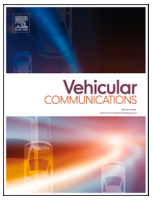
Vehicular Communications 59 (2026) 101017



Contents lists available at ScienceDirect

Vehicular Communications

journal homepage: www.elsevier.com/locate/vehcom



LAERS: Load-aware efficient routing scheme in hierarchical software defined vehicular networks

Madhuri Malakar [a,*], Judhistir Mahapatro [a], K.L.K. Sudheera [b], Bibhudatta Sahoo [b]

[a] Department of Computer Science and Engineering, National Institute of Technology, Rourkela, India
[b] Department of Electrical and Information Engineering, University of Ruhuna, Sri Lanka

ARTICLE INFO

Keywords:
Load-balancing
Routing
SDN
SDVN
VANET

ABSTRACT

The integration of vehicular networks and Software-Defined Networking (SDN) has led to the development of Software-Defined Vehicular Networks (SDVN). These networks offer flexible, programmable, and distributed control capabilities. However, the highly dynamic nature of vehicular environments, characterized by high mobility and frequent topology changes, places significant challenges on the SDVN control plane. With increasing vehicle density at the data plane, some controllers experience excessive load, resulting in increased route request processing delays and flow setup delays. In contrast, other controllers remain under utilized, leading to inefficient resource use. To address this, we propose Load-Aware Efficient Routing Scheme (LAERS), based on a hierarchical three-layered control plane (l_3-HSDVN) composed of main, domain, and local controllers. In the proposed scheme, the load status of each controller is continuously monitored, and route requests from overloaded controllers are dynamically redirected to underutilized ones. This load-balancing mechanism improves control plane efficiency, reduces operational delays in flow rule generation and installation, minimizes controller response time, and enhances routing performance at the data plane. Simulation results show that LAERS significantly outperforms existing routing schemes based on l_2-HSDVN and l_3-HSDVN, achieving improvements of up to 54.46% and 19.93% in average throughput, 52.77% and 24.28% in packet delivery ratio, and 56.48% and 21.87% in average end-to-end delay, respectively.

1. Introduction

Vehicular networks play a key role in improving the performance of Intelligent Transportation Systems (ITS), but they face persistent challenges because of frequent topology changes, unstable wireless links, and high node mobility, which collectively degrade communication reliability and routing efficiency [1]. To work around these problems, the concept of Software-Defined Networking (SDN) [2] has been integrated into vehicular environments, resulting in the Software-Defined Vehicular Networks (SDVN) paradigm. When vehicular networks are designed using SDN, a logically centralized control plane offers significant benefits such as programmability, scalability, and flexibility [3–5]. A typical SDVN with a logically centralized main controller provides several practical benefits, such as simplified network management through software updates, dynamic configuration, and service provisioning based on user demands and network conditions [6–11]. However, it suffers from operational delays,[1] increased controller load,[2] higher flow setup delays, and scalability issues with increasing vehicle density. Additionally, the need for intervention of controllers every time a routing decision is to be taken imposes a heavy burden on the logically centralized main control plane [12]. Furthermore, a single controller may struggle to provide uniform service to each vehicle and handle the massive influx of route requests as vehicle density increases in the network [13–15]. Such network instances lead to controller overload, longer response times, and degraded routing performance in SDVN [16–18].

In parallel with these challenges, security concerns in vehicular environments has emerged as an important complementary issue, where recent studies highlight the need for advanced detection mechanisms [19]; however, such security considerations are mentioned here only to contextualize the broader operational implications of SDVN control-plane traffic. A two-layered control plane architecture, known as l_2-HSDVN, is introduced to bring the control plane closer to the data plane. This architecture divides the control plane into two layers: the first layer consists of RSUs acting as local controllers positioned close to the vehicles, while

* Corresponding author.
E-mail addresses: malakar.madhuri2018@gmail.com (M. Malakar), mahapatroj@nitrkl.ac.in (J. Mahapatro), kushan@eie.ruh.ac.lk (K.L.K. Sudheera), bdsahu@nitrkl.ac.in (B. Sahoo).

[1] Operational delay refers to the delay involved in processing the route request, including flow rule generation at the controller, link transmission delay, and flow installation delay at the forwarding devices.

[2] Load at the controller refers to the number of route requests arriving at the controller requesting flow rules.

https://doi.org/10.1016/j.vehcom.2026.101017
Received 11 September 2025; Received in revised form 13 February 2026; Accepted 24 February 2026
Available online 26 February 2026
2214-2096/© 2026 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

the second layer features a centralized main controller, which may be located remotely, such as within the Internet [12,20–26].

While l_2-HSDVN, a two-layered hierarchical SDVN, alleviates centralization bottlenecks, it may also suffer from extended route discovery delay, controller overhead, scalability issues, and delayed controller response time [6]. These limitations arise because, even though the control plane is partially decentralized, the two-layer model still concentrates substantial decision-making at the main controller, which can become congested under high vehicle density. Introducing an intermediate control layer between the local and main controllers can logically distribute the control tasks, improving scalability and reducing exchange of control message congestion. Similar hierarchical layering principles have been effectively applied in other network domains to mitigate coordination overhead and balance workloads, as discussed in the study [27]. Motivated by these insights, we extend the hierarchical control concept in SDN to vehicular networks by proposing a three-layer control architecture, termed l_3-HSDVN, as depicted in Fig. 1, which is aimed at balancing the overall load across the controllers in different control-plane layers. Additionally, we also explored a multilayered control plane inspired from SDN [28], for vehicular networks. Notably, Fu et al. [29,30] states that introducing a domain control layer between the local and main control plane reduces control plane computation complexity from super-linear to linear and can significantly improve load distribution, controller overhead, ensures shorter and more efficient routing paths, minimizes communication overhead by aggregating network views at different layers, and reduces unnecessary message exchanges. Similarly, Zhao et al.[31] states that a hierarchical control plane ensures efficient control plane load distribution by distributing the controller workload across multiple control layers and prevents an exponential increase in computational complexity by managing inter-domain traffic and intra-domain traffic efficiently. Likewise, Genge et al. [32] further emphasized that the hierarchical control plane prevents the overloading of a single controller by efficiently distributing the load among multiple controllers among multiple control planes, thereby improving controller responsiveness by reducing unnecessary message exchanges. Motivated by these findings, we extend the hierarchical control concept in SDN to vehicular networks by proposing a three-layer control architecture, termed l_3-HSDVN, as depicted in Fig. 1, which is aimed at balancing the overall load across the controllers in different control plane layers.

The l_3-HSDVN introduces a domain control plane between the local and main control planes. It distributes the control responsibilities across local, domain, and main controllers, aiming to reduce operational latency and improve network scalability. Despite this architectural improvement, timely flow rule generation and dynamic controller load management remain a critical challenge in l_3-HSDVN. As discussed, controller overload under high vehicular density leads to longer response times and degraded routing performance, making timely load monitoring and redistribution essential. The proposed scheme, LAERS, considers controller load during routing, where each controller dynamically assesses its load using response time as a metric and classifies itself as underloaded, neutrally loaded, or overloaded. If a controller is overloaded, the routing request is redirected to an underloaded controller to improve the flow setup delay and reduce operational delay. Notably, when the domain control layer is excluded from the l_3-HSDVN, the architecture reverts to the l_2-HSDVN. Unlike previous hierarchical routing schemes that primarily rely on static or periodically optimized control-layer configurations, LAERS introduces a real-time, load-aware redirection mechanism that continuously adapts controller assignment decisions based on instantaneous response-time metrics. This approach shifts the focus from structural hierarchy to adaptive intelligence, where routing decisions are dynamically influenced by the current operational state of each controller rather than pre-defined topology parameters. LAERS effectively mitigates transient controller congestion and maintains stable routing performance under varying vehicular densities and mobility conditions. This adaptive load-management philosophy parallels emerging trends in intelligent network orchestration [33], to improve responsiveness and scalability in distributed vehicular environments. The proposed scheme hierarchically distributes load across the control plane, reduces operational latency during route request processing, manages overloaded controllers, and ensures improved controller response time, thereby improving flow setup delay and overall routing performance in HSDVN. To further validate the proposed scheme, we develop mathematical models in Section 3. Although the framework supports general n-layer HSDVN modeling, the analysis reveals that three control layers yield the minimum expected delay. The scheme is implemented using an NS-3 simulator and evaluated against l_2-HSDVN and l_3-HSDVN without load-handling, using routing performance metrics under varying vehicle density and speeds, detailed in Section 5.

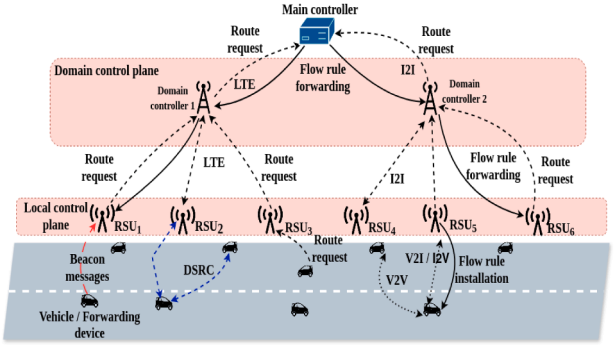


Fig. 1. The l_3-HSDVN framework showing the hierarchical control structure consisting of local controllers managing vehicle-level routing, domain controllers coordinating regional decisions among multiple locals, and the main controller oversees global policies and inter-domain synchronization.

1.1. Motivation

SDVNs offer numerous advantages for ITS, particularly in enabling smart traffic management applications. They allow vehicles to dynamically adapt to varying traffic conditions, including both sparse and dense network scenarios. This adaptability helps optimize traffic flow, reduce routing delays, and improve fuel efficiency and emissions, thereby contributing to sustainable transportation systems [34].

An efficient control plane is essential to manage routing requests and fully realize these benefits. However, an overloaded controller can degrade SDVN performance by increasing latency in flow rule generation [35]. To address this, it is necessary to manage controller load to avoid prolonged response times. In scenarios where some controllers are overloaded, traffic can be rerouted to less congested controllers, balancing the load and enabling faster processing of route requests and data packet routing. Such load-balancing mechanisms are critical for safety-related applications and enhance overall network reliability. These considerations motivate the need for load-aware and efficient routing schemes in SDVNs.

1.2. Our contribution

The contributions of our work are as follows:

- We study the existing routing schemes that use typical SDVN and l_2-HSDVN frameworks, and highlight their limitations, such as load at the main controller, increased operational delay while processing the route requests, and the additional time required for route discovery. To tackle such challenges, we introduce a l_3-HSDVN. This approach hierarchically handles the load at the control plane, improving the operational delay involved in generating and installing flow rules and enabling faster processing of route requests.
- We propose a load-aware efficient routing scheme, “LAERS”, that uses the l_3-HSDVN framework and effectively handles overloaded

Table 1
Comparison of routing schemes in SDVN highlighting key architectural features and limitations.

Author(s)	Fuzzy Logic / AI	Predictive Modeling	Load Balancing	Centralized Control	Computation Overhead	Scalability
Zhao et al. (2022) [36]	✓	✗	✗	✓	✓	✗
Ghao et al. (2021) [8]	✗	✓	✗	✓	✓	✗
Lin et al. (2022) [37]	✓	✗	✗	✓	✓	✗
Ghafoor & Koo (2017) [21]	✓	✓	✗	✓	✓	✗
Abugabah et al. (2020) [39]	✗	✗	✓	✓	✓	✗
Sudheera et al. (2019) [25]	✗	✗	✗	✓	✓	✗
Zhao et al. (2021) [38]	✓	✓	✗	✓	✓	✗

controllers at the control plane. The load at each controller is evaluated based on its response time and is categorized as under, over, or neutrally loaded. If a controller is determined to be overloaded, the system selects the minimum loaded controller from the available set of controllers and redirects the route request to it. This approach improves the responsiveness of the control plane, allowing faster processing of route requests and enhancing overall network throughput.

- We provide mathematical models for capturing the flow setup delay, queuing behavior, controller load computation, expected effective delay, and response time across an arbitrary number of control layers. Through analytical evaluation, we demonstrate that while the proposed framework generalizes to any n-layer control plane configuration, a three-layered structure minimizes the expected effective delay.
- Our proposed scheme was simulated using the NS-3 simulator and compared with l_2-HSDVN and l_3-HSDVN. The evaluation demonstrates significant improvements in packet delivery ratio, average throughput, and end-to-end delay.

The subsequent sections of the manuscript are framed as follows: Section 2 provides a relevant literature survey, followed by a detailed presentation of the system architecture and associated mathematical models for flow setup delay, load, and response time analysis in Section 3. Section 4 introduces the proposed routing scheme. Section 5 evaluates its performance through simulations, and Section 6 concludes the manuscript.

2. Related work

This section presents an in-depth exploration of existing routing and load balancing schemes in SDVN. It highlights their methodologies, advantages, and limitations, with a particular emphasis on the role of the control plane in improving network performance and scalability.

2.1. Routing schemes in SDVN

Recent research in SDVN has introduced various routing schemes to improve the packet delivery ratio and end-to-end delay. Zhao et al. [36] and Lin et al. [37] proposed fuzzy logic-based routing frameworks, augmented with reinforcement learning to improve link quality estimation and system adaptability to dynamic vehicular networks. Probabilistic models such as hidden Markov models (HMMs) [8] and neural networks [38] have been used for predictive routing and link stability forecasting. The cognitive and hybrid schemes of Ghafoor and Koo [21] and Abugabah et al. [39] incorporate spectrum sensing with centralized SDN controllers for dynamic route selection and traffic balance. Meanwhile, the optimization-based framework by Sudheera et al. [25] focuses on formulating routing as a minimum-cost capacitated flow problem to optimize latency and throughput. Table 1 presents a comparative analysis of the schemes discussed.

Despite the advancements in the existing routing schemes, several limitations persist. Most schemes rely heavily on centralized SDN controllers, introducing potential bottlenecks under high vehicle density. Approaches involving fuzzy logic, HMMs, and deep learning increase computational overhead, adversely affecting real-time routing performance. Scalability also remains a concern due to frequent topology changes, infrastructure constraints, and the absence of integrated load-balancing mechanisms. Addressing these research gaps is crucial for improving the overall routing efficiency and ensuring real-time responsiveness in future SDVN deployments.

2.2. Load-balancing schemes in SDVN

Several load-balancing schemes have been explored in the context of SDVN [40–46]. However, many of these schemes inherit the conventional SDN network environment primarily designed for static network topologies. In contrast, SDVN involves mobile vehicles that act as switches, introducing challenges related to dynamic topologies and real-time control plane load management. A comparative study of the differences between SDN and SDVN load-balancing schemes is presented in Malakar et al. [47]. Gao et al. [48] proposed a hierarchical routing scheme with load balancing for SDVN. The scheme employs a hierarchical geography routing, which refers to the three-step process involving grid, lane, and relay node selection based on grid vehicle density, vehicle transfer probability, path cost function, and node utility function. Alouache et al. [49] proposed a fuzzy logic-based adaptive load balancing scheme for blockchain-integrated SDVN. The architecture uses a multi-level blockchain system to enable dynamic reconfiguration of the blockchain based on the current network state, enhancing large-scale deployment capabilities. Mendiboure et al. [50] put forth a load and mobility-aware flow rule management scheme for SDN-based vehicular access networks. The approach emphasizes proactive flow rule deployment using metrics such as flow table utilization, vehicular dynamics, and control channel load, thereby optimizing network performance.

While [48] and [50] represent important hierarchical load-balancing efforts in SDVN, both depend on static or periodically triggered reconfiguration processes that restrict their responsiveness under highly dynamic vehicular conditions. In contrast, the proposed LAERS framework adopts an event-driven, response-time-based load-redirection strategy that continuously adjusts controller assignments in real time without requiring costly relocation or optimization cycles. This design provides faster adaptation to transient load fluctuations and network mobility, enhancing control-plane stability and scalability within the existing hierarchical architecture. Hence, LAERS fundamentally differs from prior hierarchical models by embedding continuous, feedback-driven decision making directly into the controller interaction process. In addition to these hierarchical and load-balancing approaches, recent works have explored AI-assisted and reinforcement-learning-based routing strategies for SDVN and VANETs [33]. These schemes primarily optimize data-plane forwarding decisions through predictive modeling or adaptive path selection, rather than addressing control-plane congestion or controller response-time management. Since LAERS focuses on real-time controller load mitigation within a hierarchical control architecture, AI/RL-based routing schemes operate on a different functional layer and are therefore not directly comparable to the control-plane-oriented contributions of this work.

A comparative study of the load-balancing schemes in SDVN is presented in Table 2. Additionally, we present a comprehensive study of

Table 2
Comparison of load-balancing schemes in SDVN, highlighting the diversity of existing strategies.

Author(s)	Vehicle Awareness	Hierarchical/Adaptive	Fuzzy / AI	Flow Mgmt.	Scalable
Gao et al. (2018) [48]	✓	✓	✗	✗	✓
Alouache et al. (2022) [49]	✓	✓	✓	✗	✓
Mendiboure et al. (2020) [50]	✓	✗	✗	✓	✗

SDVN schemes based on control and performance metrics in Table 3. Furthermore, Table 4 summarizes recent SDVN and VANET routing and load-balancing frameworks, highlighting their key features and positioning the proposed LAERS framework within ongoing research. The identified limitations emphasize the need for a well-structured, hierarchical system architecture to support control plane operations tailored to SDVN's unique challenges. Accordingly, Section 3, details the proposed SDVN system architecture, detailing its layered design, control mechanisms, and the inter-component interactions.

3. System architecture

This section presents the system model of l_3-HSDVN, the foundational architecture for our proposed scheme, comprising a hierarchical control plane and a distinct data plane, provides the flow setup delay model detailing how requests propagate from the data plane to the hierarchical control plane and experience delays at various layers. Additionally, the load at each controller within the control plane and the average response time across the hierarchical layers are computed to examine how varying numbers of route requests influence the overall routing performance in HSDVN.

3.1. System model

The l_3-HSDVN is structured into two distinct planes: the control and data planes.

1) *Control plane*: The control plane adopts a hierarchical approach and is segmented into three interconnected layers: the main, domain, and local control plane, each consisting of their respective controllers as presented in Fig. 1. These controllers act as decision-makers and generate flow rules for routing the data packets from the source to the destination vehicle while processing the routing request [3]. The seamless integration and effective coordination across inter-layer communication, i.e., I2I communication, uses Long-Term Evolution (LTE). The multiple local controllers handle localized routing decisions for vehicles within their immediate coverage areas; domain controllers manage broader geographic regions by aggregating information from several local controllers; and a single, logically centralized main controller coordinates overall network strategy and decision-making. These hierarchically arranged controllers at the control plane collaboratively generate flow rules that facilitate efficient routing of data packets from source to destination vehicles, processing routing requests promptly.

Let us consider the HSDVN architecture, where the control plane can be mathematically modeled as a three-layer hierarchy comprising the local, domain, and main controllers, denoted as control planes l_1, l_2, and l_3. Let $c_l = \{c_{l_1}, c_{l_2}, \ldots, c_{l_n}\}$ represent the set of n local controllers, $c_d = \{c_{d_1}, c_{d_2}, \ldots, c_{d_m}\}$ denote the set of m domain controllers, and c_m represent the single main controller. Let the set of all controllers across all control planes be denoted as $C = \{c_l \cup c_d \cup c_m\}$, with cardinality as $|C| = n + m + 1$. Each local controller $c_{l_i} \in l_1$ manages a subset of vehicles within its coverage area, defined by $\mathcal{V}_i = \{v_{i,1}, v_{i,2}, \ldots, v_{i,k}\}$, where, $v_{i,j}$ denotes the jth vehicle associated with local controller c_{l_i}. The elements of data and the control planes are associated with each other using existing association policies such as χ [56] such that $\chi : X \rightarrow Y$:

- If $X = \{v_{i,j}\}$ and $Y = \{c_{l_i}\}$, then $\chi(v_{i,j}) = c_{l_i}$; ▷ jth vehicle is associated with ith local controller c_{l_i}
- If $X = \{c_{l_i}\}$ and $Y = \{c_{d_j}\}$, then $\chi(c_{l_i}) = c_{d_j}$; ▷ ith local controller is associated with a jth domain controller
- If $X = \{c_{d_j}\}$ and $Y = \{c_m\}$, then $\chi(c_{d_j}) = c_m$; ▷ jth domain controller is associated with the main controller

The association policies outlined above establish a hierarchical mapping between vehicles and controllers across the control plane layers. These mappings ensure that each vehicle is governed by a specific local controller, which in turn is managed by a domain controller, all under the coordination of the main controller. Building upon these associations, each local controller generates flow rules defining vehicle routing paths within its coverage area. Let the local controllers generate flow rules $f_{c_{l_i}}$ for its associated vehicles be defined as:

$$f_{c_{l_i}} = \{f_{(v_s, v_d)} | v_s, v_d \in v_i\} \tag{1}$$

where, $f_{(v_s, v_d)}$ represents a flow rule from source vehicle v_s to destination vehicle v_d.

Once the local controllers generate flow rules for routing traffic within their immediate domains, this information must be escalated to the next level in the hierarchy to maintain coordinated control across wider geographic regions. This is achieved through the domain controllers, which serve as intermediate decision-making entities responsible for aggregating flow rules from their associated local controllers. The domain controllers in l_2 aggregate flow rules from multiple local controllers. Let the domain-level flow rules $f_{c_{d_j}}$ generated by domain controller c_{d_j} be defined as:

$$f_{c_{d_j}} = \bigcup_{c_{l_i} \in c_{d_j}} f_{c_{l_i}} \tag{2}$$

The aggregated flow rules at the domain controllers provide a regional view of the network, enabling intermediate-level routing decisions. The main controller uses these domain-level flow rules to achieve end-to-end visibility and generate global routing policies. As the top-tier decision-maker, the main controller synthesizes information from all domain controllers to generate flow rules formulated as follows:

$$f_{c_m} = \bigcup_{c_{d_j} \in c_d} f_{c_{d_j}} \tag{3}$$

2) *Data plane*: The data plane, denoted as l_0, forms the foundational layer of HSDVN architecture. It comprises a set of vehicle nodes represented by $\mathcal{V}$. Each vehicle $v_i \in \mathcal{V}$ is equipped with an On-Board Unit (OBU) [57], which operates as an SDN-compliant forwarding device that follows the flow rules and instructions issued by the control plane. The vehicles in the data plane support Vehicle-to-Vehicle (V2V) communication that allows direct data exchange between neighboring vehicles v_i and v_j, and Vehicle-to-Infrastructure (V2I) communication enables interaction between a vehicle v_i and its associated local controller $c_{l_j} \in c_l$, which serves as a roadside infrastructure unit using Dedicated Short-Range Communications (DSRC) [58]. These communication modes collectively facilitate dynamic routing and real-time data dissemination across the network, as illustrated in Fig. 1. To support adaptive routing, each vehicle v_i periodically broadcasts a beacon message b_i, defined as:

$$b_i = (ID_i, GPS_i, Speed_i, t_i) \tag{4}$$

where ID_i is the vehicle's identifier, GPS_i represents its geographic location, $Speed_i$ is its instantaneous velocity, and t_i denotes the timestamp of the broadcast [59]. These beacon messages are received by the associated local controllers c_{l_j} through V2I communication links. The control plane leverages the received beacon data to construct hierarchical network graphs. Upon receiving beacon messages, each local controller c_{l_j}

Table 3
Comprehensive comparison of SDVN schemes categorized by controller architecture and evaluated against key control and performance metrics.

Group →	Centralized controller			Local controllers							Miscellaneous					
Category\Author(s)	Zhu et al. [4]	Zhu et al. [9]	Liu et al. [10]	Ji et al. [7]	He et al. [51]	Zhao et al. [8]	Ghafoor et al. [21]	Chahal et al. [22]	Dong et al. [23]	Kazmi et al. [24]	Sud-heera et al. [25]	Sud-heera et al. [26]	Oubatti et al. [17]	Correia et al.[52]	Gao et al. [48]	Alouache et al. [49]	Mendi-boure et al. [50]
High operational latency	✓	✓	✓	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
Multiple controllers	✗	✗	✗	✗	✗	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Controller load	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
Vehicle load	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗
Hierarchical routing	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗
load-balancing scheme	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗

builds a localized network graph based on the vehicles within its coverage. These local graphs are then forwarded to the respective domain controllers $c_{d_k} \in c_d$, which aggregate the information to form domain-level control graphs. Finally, these domain graphs are transmitted to the main controller c_m, which synthesizes them into a global network graph $G = (\mathcal{V}, \varepsilon)$, where ε denotes the set of communication links. This hierarchical graph ensures that routing decisions are efficiently coordinated and dynamically updated based on real-time vehicular topology.

Upon receiving a route request from a source vehicle $v_s \in \mathcal{V}$, the corresponding controller c_{l_j}, or higher-level controllers as necessary, processes the request by utilizing the available graph information, computes an optimal route, and generates a flow rule set f_{v_s} tailored for the source vehicle. These flow rules are transmitted back to the vehicles through V2I links, enabling the source vehicle v_s to forward data packets toward the destination vehicle v_d through multi-hop V2V communication paths in strict accordance with the control plane's instructions. Through this tightly integrated interaction with the control plane, the data plane ensures efficient, scalable, and adaptive forwarding operations in the HSDVN framework. While the data and control planes jointly manage routing and forwarding, hierarchical coordination inherently introduces multi-level communication overhead. As discussed in Section 1, these latencies primarily arise from inter-controller interactions and flow setup processes. To analytically capture their impact on control-plane responsiveness, the subsequent Section 3.2 develops a hierarchical flow-setup delay model across the control layers.

3.2. Hierarchical flow setup delay model in HSDVN

This section introduces a hierarchical flow setup delay model to evaluate the responsiveness and efficiency of l_3-HSDVN by considering the cumulative impact of transmission, propagation, queuing, and processing delays across the control plane [56]. Each vehicle-initiated routing request undergoes an uplink phase to a control plane, processing at the controller, and a downlink response phase. The queuing delay at each controller is dynamically modeled based on vehicle density, where the arrival rate is proportional to the number of active vehicle nodes. The total delay at the control planes is compactly represented using a vectorized formulation, and the final flow setup delay is modeled by computing mathematical formulation of delay components, cumulative delay at control plane layers, expected flow setup delay, and expected effective flow setup delay as follows.

3.2.1. Mathematical formulation of delay components

For any two consecutive layers $l_i \rightarrow l_{i+1}$, the delay component is composed of uplink ($\delta^u_{l_i,l_{i+1}}$), queuing ($\delta^q_{l_i}$), processing ($\delta^{pr}_{l_{i+1}}$), and downlink ($\delta^d_{l_{i+1},l_i}$) delays. These delays are computed as follows:

$$\delta^u_{l_i,l_{i+1}} = \frac{S}{B} + \frac{d_{i,i+1}}{s} + \frac{1}{\mu_{l_{i+1}} - \lambda_{l_{i+1}}} \tag{5}$$

$$\delta^{pr}_{l_{i+1}} = \frac{\omega_{l_{i+1}}}{\Omega_{l_{i+1}}} \tag{6}$$

$$\delta^d_{l_{i+1},l_i} = \frac{S}{B_{i+1,i}} + \frac{d_{i+1,i}}{s} \tag{7}$$

where S: packet size, B: bandwidth, $d_{i,j}$: distance, s: is the speed of light, μ: service rate, λ: arrival rate, ω: the number of CPU cycles required for processing Packet-In messages, and Ω: the processing capacity of the node respectively. In the uplink delay component, the queuing delay experienced at each controller plays a crucial role. Since the number of routing requests inherently depends on vehicle density, it is essential to model the arrival rate dynamically as a function of the number of active vehicles in the network. The arrival rate at each controller depends dynamically on the number of active vehicles and can be expressed as:

$$\lambda_{l_i} = \alpha_i N_v \tag{8}$$

Table 4
Comparison of recent routing and load-balancing schemes.

Author(s)	Load Balancing	Hierarchical Control	Real-Time Adaptation	SDVN-Oriented	AI/ML-Based
Yusuf et al. [53]	✓	✗	✗	✗	✗
Al-Inizi et al. [1]	✗	✗	✗	✗	✗
Lin et al. [37]	✓	✗	✓	✓	✗
Gopalakrishnan et al. [33]	✗	✗	✗	✗	✓
Ghasab et al. [54]	✗	✗	✓	✗	✗
Malakar et al. [47]	✓	✗	✗	✓	✗
Hussein et al. [11]	✗	✗	✓	✓	✗
Al-Ibraheemi et al. [55]	✗	✗	✗	✗	✓
Robert et al. [27]	✗	✓	✗	✗	✓
Liu et al. [20]	✓	✗	✗	✗	✗
LAERS	✓	✓	✓	✓	✗

where α_i is the request generation rate and N_v is the number of active vehicle nodes. Thus, the queuing delay is computed as:

$$\delta^q_{l_i} = \frac{1}{\mu_{l_i} - \alpha_i N_v} \tag{9}$$

Having individually modeled the transmission, queuing, propagation, and processing delays between control layers, it becomes necessary to aggregate these components to compute the total delay experienced during the route setup process. Thus, we present the cumulative formulation of delays as a request traverses through multiple hierarchical control layers.

3.2.2. Cumulative delay at each control plane layer

The total cumulative delay experienced when a routing request is processed at a particular control plane layer l_k is given by the sum of delays over all preceding transitions from the data plane l_0 up to l_k. Formally, the cumulative delay can be expressed as:

$$\delta_{l_k} = \sum_{i=0}^{k-1} \left(\delta^u_{l_i,l_{i+1}} + \delta^q_{l_{i+1}} + \delta^{pr}_{l_{i+1}} + \delta^d_{l_{i+1},l_i} \right) \tag{10}$$

Here, l_0: data plane, $l_1, l_2, \ldots, l_k$: progressively higher control layers, $\delta^u_{l_i,l_{i+1}}$: uplink transmission and propagation delay from layer l_i to l_{i+1}, $\delta^q_{l_{i+1}}$: queuing delay experienced at controller l_{i+1}, $\delta^{pr}_{l_{i+1}}$: processing delay at controller l_{i+1}, $\delta^d_{l_{i+1},l_i}$: downlink transmission and propagation delay from l_{i+1} back to l_i. This hierarchical summation captures the cumulative cost incurred as a routing request traverses multiple layers, accounting for communication and computational delays at each control node.

While the cumulative delay δ_{l_k} provides the total setup delay when a request is processed at a particular control plane layer, in HSDVN, a request may succeed at different layers depending on network conditions. Therefore, it is useful to represent the cumulative delays across all control layers as a delay vector. This vectorized representation enables efficient computation of the overall flow setup delay based on the specific layer where the routing decision is made. Thus, the delay vector is represented as:

$$\boldsymbol{\delta} = \begin{bmatrix} \delta_{l_1} \\ \delta_{l_2} \\ \vdots \\ \delta_{l_n} \end{bmatrix} \tag{11}$$

Where δ_{l_i} represents the cumulative delay associated with control plane layer l_i, ordered hierarchically from the local control plane up to the main controller. Let the flow decision vector be represented as:

$$\mathbf{f} = \begin{bmatrix} f_{l_1} \\ f_{l_2} \\ \vdots \\ f_{l_n} \end{bmatrix} \quad \text{where} \quad f_{l_i} \in \{0,1\}, \quad \sum_{i=1}^{n} f_{l_i} = 1 \tag{12}$$

Thus, the total flow setup delay can be computed as:

$$\delta_{\text{total}} = \mathbf{f}^T \boldsymbol{\delta} \tag{13}$$

While the vectorized formulation provides the deterministic flow setup delay, assuming successful processing at a particular control layer, real-world network dynamics introduce uncertainty. Depending on network congestion and controller processing load, routing requests may succeed at different control layers with varying probabilities. To realistically capture the flow setup behavior, it is necessary to incorporate success probabilities into the delay modeling, leading to an expected flow setup delay formulation.

3.2.3. Expected flow setup delay with success probabilities

Let $p_{l_1}, p_{l_2}, \ldots, p_{l_n}$ denote the success probabilities at layers $l_1, l_2, \ldots, l_n$, respectively, with $p_{l_n} = 1$, assuming that the main controller always succeeds. The expected total flow setup delay is given by:

$$\mathbb{E}[\delta_{\text{total}}] = \sum_{k=1}^{n} \left(\prod_{i=1}^{k-1} (1 - p_{l_i}) \right) p_{l_k} \delta_{l_k} \tag{14}$$

Where, $\prod_{i=1}^{k-1}(1 - p_{l_i})$ represents the cumulative probability that the request fails at all preceding layers $l_1, l_2, \ldots, l_{k-1}$, p_{l_k} represents the probability that the request is successfully processed at layer l_k, δ_{l_k} represents the cumulative delay experienced if the request is processed at control plane layer l_k.

The deterministic flow setup delays δ_{total} represents the total time required to establish a route when the routing decision is successfully made at a specific control plane layer, corresponding to a known and fixed flow decision vector $\mathbf{f}$. However, in real-world vehicular network environments, routing success at a particular layer is probabilistic due to network congestion, variable service rates, and dynamic vehicular mobility. Therefore, it becomes necessary to model the flow decision $\mathbf{f}$ as a random variable rather than a deterministic choice. Consequently, the expected flow setup delay $\mathbb{E}[\delta_{\text{total}}]$ is defined as the expected value of the deterministic flow setup delay across all possible flow decision outcomes, weighted by the probability of successful processing at each control plane layer. Mathematically, this relationship can be expressed as:

$$\mathbb{E}[\delta_{\text{total}}] = \mathbb{E}_{\mathbf{f}} \left[\mathbf{f}^T \boldsymbol{\delta} \right] \tag{15}$$

Where $\mathbb{E}_{\mathbf{f}}$ denotes the expected value of deterministic Flow Setup Delay across all flow decision realizations weighted by success probabilities, and the probability distribution of $\mathbf{f}$ is determined by the layer-wise success probabilities.

3.2.4. Penalty for delay escalation

In HSDVN, if a routing request is not successfully processed at lower control layers, it must escalate to higher-level controllers. This escalation incurs delays beyond the normal communication and processing times, primarily due to increased control signaling overhead, more complex decision-making processes, and longer processing paths. Therefore,

to realistically account for the additional latency introduced by upward escalations, a penalty term is incorporated into the flow setup delay model. Let π denote the penalty incurred for each escalation to a higher control layer. The total escalation penalty associated with a routing request is defined as:

$$\text{Penalty} = \pi \times \left(\sum_{k=2}^{n} (k-1) f_{l_k} \right) \tag{16}$$

where, $f_{l_k} \in \{0, 1\}$ indicates whether the request is processed at layer l_k, $(k-1)$ represents the number of escalations required to reach layer l_k from the data plane. Thus, the effective total flow setup delay, including the escalation penalty, is given by:

$$\delta_{\text{effective}} = \delta_{\text{total}} + \text{Penalty} \tag{17}$$

Where δ_{total} represents the cumulative delay calculated from the deterministic flow decision or its expected value across probabilistic outcomes.

3.2.5. Expected effective flow setup delay

While the effective flow setup delays $\delta_{\text{effective}}$ captures both routing delay and escalation penalties in a deterministic setting, real-world vehicular networks introduce uncertainty in where the routing request will be successfully processed. Thus, it becomes essential to model the expected effective flow setup delay by combining success probabilities and escalation penalties. The expected effective flow setup delay $\mathbb{E}[\delta_{\text{effective}}]$ can be expressed as:

$$\mathbb{E}[\delta_{\text{effective}}] = \mathbb{E}[\delta_{\text{total}}] + \mathbb{E}[Penalty] \tag{18}$$

where,

$$\mathbb{E}[\text{Penalty}] = \pi \times \mathbb{E}\left[\sum_{k=2}^{n} (k-1) f_{l_k} \right] \tag{19}$$

This leads to the formulation:

$$\mathbb{E}[\text{Penalty}] = \pi \mathbb{E}[E] \tag{20}$$

Where $\mathbb{E}[E]$ is the expected number of escalations, given by:

$$\mathbb{E}[E] = \sum_{k=1}^{n} (k-1) \left(\prod_{i=1}^{k-1} (1 - p_{l_i}) \right) p_{l_k} \tag{21}$$

Thus, the expected effective flow setup delay can be expressed as:

$$\mathbb{E}[\delta_{\text{effective}}] = \mathbb{E}[\delta_{\text{total}}] + \pi\, \mathbb{E}[E] \tag{22}$$

3.3. Optimal number of control layers

The computed delay model shows that the expected effective flow setup delay $\mathbb{E}[\delta_{\text{effective}}]$ depends intricately on the structure of the control plane hierarchy, particularly the number of control layers n. A smaller number of control layers results in fewer escalation steps but can lead to higher queuing delays due to each controller's overload. Conversely, a larger number of layers reduces queuing delays but introduces higher escalation penalties and processing overheads. Thus, a trade-off exists between minimizing queuing delay and minimizing escalation penalties. To achieve optimal performance, it becomes necessary to determine the optimal number of hierarchical control layers n^* that minimizes the expected effective flow setup delay.

3.3.1. Optimal number of layers

The optimal number of control layers n^* minimizes the effective delay $\delta_{\text{effective}}(n)$. The total delay generally shows a U-shaped behavior concerning n. For small n, high queuing delay due to overloading, whereas for large n, high escalation and processing delay overhead occurs. Thus, the optimization condition is to find n^* such that:

$$n^* = \arg\min_n \delta_{\text{effective}}(n) \tag{23}$$

Practically, this can be determined by evaluating $\delta_{\text{effective}}(n)$ for varying n and selecting the n with the minimum delay. Thus, mathematically,

$$\frac{d}{dn} \delta_{\text{effective}}(n) = 0 \quad \text{and} \quad \frac{d^2}{dn^2} \delta_{\text{effective}}(n) > 0 \tag{24}$$

The first condition ensures a stationary point, and the second condition guarantees that the stationary point corresponds to a minimum. Further, the proof of optimality is provided in Appendix A.

While the preceding section thoroughly analyzed the hierarchical flow setup delay in HSDVN, considering multiple delay components, it is important to recognize that these delays are intrinsically linked to the dynamic load experienced by each control layer. As vehicular density and route request rates varies, the load imposed on local, domain, and main controllers fluctuates accordingly. These variations in load not only influence queuing and processing delays but also impact routing success probabilities and escalation patterns. Therefore, to gain a more holistic understanding of system performance and to support delay-awareness, the subsequent Section 3.4, investigates the computation and modeling of control plane load in HSDVN.

3.4. Load computation

In a typical HSDVN setup, routing requests generated by vehicles are aggregated at the ingress of the control plane. These requests form queues and are processed sequentially through multiple controller layers. The queue at each controller follows the M/M/1 model, assuming Poisson-distributed arrivals and exponentially distributed service times. Let N_v denote the number of active vehicle nodes, α is the average rate at which a vehicle generates route requests per second, and n is the number of control layers. Each control layer has a service rate μ_i and a routing success probability β_i. Let the request arrival rate at layer l_1 be given as:

$$\lambda_{l_1} = N_v \times \alpha \tag{25}$$

If the processing of a route request fails at l_1 with probability $(1 - \beta_1)$, requests escalate to the next layer. Therefore, the arrival rate at layer l_2 can be mathematically formulated as:

$$\lambda_{l_2} = (1 - \beta_1) \times \lambda_{l_1} \tag{26}$$

This escalation continues through the hierarchy, with the generalized arrival rate at any layer l_k:

$$\lambda_{l_k} = \left(\prod_{i=1}^{k-1} (1 - \beta_i) \right) \lambda_{l_1}, \forall k = 1, 2, \ldots, n \tag{27}$$

To represent this systematically, the arrival rate can be defined as:

$$\lambda = \begin{bmatrix} \lambda_{l_1} \\ \lambda_{l_2} \\ . \\ . \\ . \\ \lambda_{l_n} \end{bmatrix} \tag{28}$$

and the corresponding escalation transformation matrix E is defined as:

$$E = \begin{bmatrix} 1 \\ 1 - \beta_1 \\ (1 - \beta_1)(1 - \beta 2) \\ . \\ . \\ . \\ \prod_{i=1}^{n-1} (1 - \beta_i) \end{bmatrix} \tag{29}$$

which leads to the compact form as

$$\lambda = E \times (N_v \times \alpha) \tag{30}$$

Given the arrival rate at each layer, the controller utilization $\rho_{l_k} = \frac{\lambda_{l_k}}{\mu_{l_k}}$ determines the load status. Controller stability requires that $\rho_{l_k} < 1$, otherwise queue length grows unbounded. The average queuing delay at

each layer, derived from the M/M/1 model, is given by $\delta^q_{l_k} = \frac{1}{\mu_{l_k} - \lambda_{l_k}}$. This delay grows sharply as λ_{l_k} approaches μ_{l_k}, indicating high congestion. To capture the overall system impact, we compute the total effective load at the control plane as follows.

$$\lambda_{total} = \sum_{k=1}^{n} \lambda_{l_k} \tag{31}$$

Building upon the load analysis, it becomes essential to understand how the system responds to the routing requests that traverse the control plane hierarchy. While load metrics quantify the pressure on each controller, they do not fully capture the time a request spends within the system from initiation to resolution. This motivates a detailed examination of the response time, encapsulating the queuing, transmission, and processing delay at each control layer, discussed in Section 3.5.

3.5. Response time computation

Let us denote t_{req} as the time the route request arrives at a particular controller node, and t_{rep} denotes the controller reply time for the requested routing path. Thus, the response time (Δt) [60] to a $packet_{in}$ route request can be computed as:

$$\Delta t_{l_k} = t_{rep,l_k} - t_{req,l_k} \tag{32}$$

or

$$\Delta t_{l_k} = \delta^q_{l_k} + \delta^{pr}_{l_k} = \frac{1}{\mu_{l_k} - \lambda_{l_k}} + \frac{\omega_{l_k}}{\Omega_{l_k}} \tag{33}$$

In vector notation,

$$\Delta t_{l_k} = \begin{bmatrix} \Delta t_{l_1} \\ \Delta t_{l_2} \\ \cdot \\ \cdot \\ \cdot \\ \Delta t_{l_n} \end{bmatrix} = \begin{bmatrix} \frac{1}{\mu_{l_1} - \lambda_{l_1}} + \frac{\omega_{l_1}}{\Omega_{l_1}} \\ \frac{1}{\mu_{l_2} - \lambda_{l_2}} + \frac{\omega_{l_2}}{\Omega_{l_2}} \\ \cdot \\ \cdot \\ \cdot \\ \frac{1}{\mu_{l_n} - \lambda_{l_n}} + \frac{\omega_{l_n}}{\Omega_{l_n}} \end{bmatrix} \tag{34}$$

The detailed analysis of delay, load, and response time in the HSDVN architecture reveals several critical performance bottlenecks. High queuing delays under load, increased escalation penalties due to controller saturation, and the probabilistic nature of routing success highlight the need for a more adaptive and scalable solution to improve the routing performance in HSDVN.In distributed infrastructures, adaptive coordination must also align with privacy-preserving operational paradigms, as demonstrated by recent federated defense approaches such as PAFA-DoH [61]. Although privacy preservation is beyond the scope of this work, such studies reinforce the broader relevance of adaptive coordination in distributed control environments.

Further, we now discuss the proposed scheme elaborately in Section 4 that improves control plane responsiveness and routing performance in HSDVN. The equilibrium response delay obtained from this model under nominal traffic load conditions is approximately 0.005 s, which is used as the baseline reference threshold for load classification in LAERS.

4. LAERS: Proposed scheme

In this section, we present the proposed routing scheme, which integrates controller load-awareness in HSDVN. The scheme builds upon the delay and load models developed in Section 3 and employs a multi-layered control architecture designed for scalability and responsiveness. Routing decisions are driven by vehicle-controller association, controller response time thresholds, and real-time load status. By incorporating mathematical thresholds derived from queueing theory and response time estimation, the scheme ensures efficient load distribution and timely flow rule generation across the control plane hierarchy.

Algorithm 1 presents flow rule generation logic for HSDVN. It operates by identifying the lowest possible control plane layer where both the source (v_s) and destination (v_d) vehicles are associated with a common controller, while also considering the controller's load status based on a predefined response time threshold r_{th}. The algorithm first attempts to resolve the request at the local control layer l_1 by retrieving the associated controllers c_s and c_d for both vehicles using the association function $\chi()$, as described in Section III-A. If both vehicles share the same local controller and its response time is below r_{th}, a flow rule is generated using the local topology G_{l_1}. If the controller is overloaded, an alternative underloaded controller is selected using Algorithm 3, which internally relies on the single-controller load classifier, Algorithm 2. If the request cannot be resolved at the local level, the algorithm ascends through higher control layers (domain, then main) following the hierarchy defined in Section 2. At each level l_i, it checks whether v_s and v_d are associated with the same controller. If such a controller is found and its response time is acceptable, it generates the required flow rule. Otherwise, it invokes Algorithm 3 to identify the underloaded associated controller with the minimum response time using an argmin selection rule, forwarding the request accordingly. This redirection process ensures that overloaded controllers are bypassed, maintaining system responsiveness. If none of the controllers at any level are suitable, the algorithm returns a null result, indicating that the request could not be processed within acceptable delay bounds. This decision logic is driven by the queuing-based response time model introduced in Section III-E and reflects the controller association framework defined in Section III-A. This redirection logic is directly supported by the analytical formulations in Section 3, where controller response time and queuing behavior are expressed as functions of load and escalation probability. LAERS thereby ensures scalable and delay-aware route discovery across the control plane hierarchy, effectively balancing load while maintaining minimal latency in dynamic vehicular environments. It is worth noting that the computational overhead of Algorithm 1 is lightweight, as the controller-level checks scale linearly with the number of controllers in each layer. Since the algorithm operates on controller metadata rather than per-vehicle state, its complexity remains independent of vehicular density, ensuring scalability for large-scale HSDVN deployments.

Algorithm 2 is a utility procedure designed to classify the load status of a every controller based on its instantaneous response time. For each controller $c \in \mathbb{C}$, the algorithm retrieves its response time r. Given a controller c, the algorithm retrieves its response time r and compares it against the predefined response threshold r_{th}, which is derived from the queuing-based delay model presented in Section III-E. If $r < r_{th}$, the controller is classified as underloaded, indicating that it can process additional route requests without incurring significant delay. If $r = r_{th}$, the controller is considered neutrally loaded, signifying it is at the boundary of its operational capacity. When $r > r_{th}$, the controller is overloaded, suggesting it should not accept new flow setups to avoid further delay accumulation. This classification reflects the practical performance limits of the control nodes within the hierarchical SDVN architecture described in Section 2. The algorithm outputs the load-state s_c of controller c along with its computed response time r_c, which serve as inputs to the controller selection mechanism implemented in Algorithm 3. It is further clarified that the response-time threshold r_{th} used for controller load classification is not a fixed or arbitrary constant. The value of 0.005 s reported in Table 5 is derived from the queuing-delay model in Section III-E, where the controller response time increases sharply as the request arrival rate λ approaches the service rate μ. This value represents the equilibrium operating point under nominal traffic load and serves as the baseline threshold for distinguishing underloaded, neutral, and overloaded states. During runtime, each controller compares its instantaneous response time against this analytically derived baseline, allowing the effective threshold to adjust naturally with variations in vehicular density and request arrival rate, thereby ensuring stable load-state decisions even under fluctuating network conditions. Regarding redirection feasibility, LAERS limits request forwarding to controllers

Table 5
Mathematical symbols and description.

Symbol	Description
l_1, l_2, l_3	Local, domain, and main control plane
c_l	Set of n local controllers
c_d	Set of m domain controllers
c_m	Main controller
C	Set of all controllers
$\lvert C\rvert$	Cardinality of set C
$\mathcal{V}$	Set of vehicles
v_s, v_d	Source and destination vehicles
χ	Association function
f	flow rule
δ^u	Uplink delay
δ^d	Downlink delay
δ^q	Queuing delay
δ^{pr}	Processing delay
λ	Request arrival rate at controller
μ	Service rate of controller
$\mathbb{E}[\delta_{\text{total}}]$	Expected flow setup delay
n	Number of control planes
Δt_{l_k}	Response time of controller l_k
ω_{l_k}	CPU cycles required to process Packet-In at layer l_k
Ω_{l_k}	Processing capacity (cycles/sec) of layer l_k

already associated with the source vehicle. This avoids excessive inter-domain control packet exchanges, prevents long I2I traversal delays, and preserves geographic locality within the hierarchical SDVN structure. Since each domain contains multiple associated controllers, redirection remains feasible without increasing routing overhead. This localized and lightweight coordination aligns with centralized-yet-adaptive control strategies discussed in [54], ensuring that load balancing remains practical and efficient across dynamic vehicular environments.Similar adaptive principles are observed in bio-inspired learning frameworks for distributed threat detection, such as Neuromorphic Quantum Adversarial Learning (NQAL) [62], that emphasize decentralized and responsive decision-making in complex infrastructures. In contrast, LAERS achieves adaptive routing through analytically derived load and response-time modeling without relying on learning-based mechanisms. It is particularly useful when making routing decisions in dynamic conditions where controller congestion must be dynamically detected and mitigated. The load categorization logic aligns directly with the thresholds used in both the LAERS algorithm and its supporting controller selection utilities. Since the procedure operates on a single controller, its complexity is $O(1)$, ensuring constant-time load-state evaluation.

Algorithm 3 is a utility function that extends the functionality of Algorithm 2 by selecting not just any underloaded controller, but the most optimal one, i.e., the associated controller with the minimum response time. Given the set of controllers C and a source vehicle v_s, the algorithm classifies each candidate controller on demand using Algorithm 2, rather than relying on precomputed load-status and response-time maps. The algorithm first filters the controllers to include only those that are associated with v_s (as determined by the association function $\chi(v_s, c)$) and whose load state is evaluated as UNDERLOADED through the single-controller classifier. Among these qualifying candidates, it then selects the controller with the lowest response time, denoted as c_{ul}, using an explicit $\arg\min_{c \in \mathcal{U}} R[c]$ selection rule, where $R[c]$ is obtained directly from Algorithm 2. If no such controllers are found, the function returns NULL, indicating that redirection is not feasible under the current system load. This algorithm is tightly coupled with the hierarchical controller association model presented in Section III-A and the queuing-based delay framework described in Section III-E. It provides a mathematically grounded mechanism for prioritizing controller selection during load-aware redirection, particularly in the LAERS algorithm. By ensuring that redirection targets the controller with the best expected responsiveness, this utility helps minimize end-to-end flow setup delay, enhance system efficiency, and maintain quality of service under dynamic vehicular traffic conditions. Because the selection is performed over a bounded controller set and involves a single pass over C with constant-time classification per controller, the utility runs in linear time and does not impact scalability.

Algorithm 1 LAERS: Load-aware flow rule generation in hierarchical control plane.

Require: Source and destination vehicles v_s, v_d; controller hierarchy: local controllers $\mathcal{L}$, domain controllers D, main controller m; response-time threshold r_{th}
Ensure: Flow rule f_{v_s} for v_s, or NULL if no controller can satisfy the delay bound

for all $d \in \mathcal{D}$ ▷ Iterate over all domain controllers
  $\mathcal{L}_d \leftarrow$ GETLOCALCONTROLLERS(d)
  for all $l \in \mathcal{L}_d$ **do** ▷ Check each local controller under domain d
    if $\chi(v_s, l) \wedge \chi(v_d, l)$ **then** ▷ Both vehicles associated with the same local controller
      $r \leftarrow$ GETRESPONSETIME(l)
      if $r \le r_{th}$ **then**
        $f_{v_s} \leftarrow$ GENERATEFLOWRULES(l, v_s, v_d)
        return $\{f_{v_s}\}$
      else
        $c_{ul} \leftarrow$ SELECTUNDERLOADEDCONTROLLER($\mathcal{L}_d, v_s, r_{th}$) ▷ Algorithm 3
        if $c_{ul} \neq$ NULL **then**
          FORWARDREQUEST($l \rightarrow c_{ul}$ via d)
          $f_{v_s} \leftarrow$ GENERATEFLOWRULES(c_{ul}, v_s, v_d)
          return $\{f_{v_s}\}$
        end if
      end if
    else if $\chi(v_s, d) \wedge \chi(v_d, d)$ **then** ▷ Vehicles associated only at the domain-controller level
      $r \leftarrow$ GETRESPONSETIME(d)
      if $r \le r_{th}$ **then**
        $f_{v_s} \leftarrow$ GENERATEFLOWRULES(d, v_s, v_d)
        return $\{f_{v_s}\}$
      else
        $c_{ul} \leftarrow$ SELECTUNDERLOADEDCONTROLLER($\mathcal{D}, v_s, r_{th}$)
        if $c_{ul} \neq$ NULL **then**
          FORWARDREQUEST($d \rightarrow c_{ul}$ via m)
          $f_{v_s} \leftarrow$ GENERATEFLOWRULES(c_{ul}, v_s, v_d)
          return $\{f_{v_s}\}$
        end if
      end if
    end if
  end for
end for
$r_m \leftarrow$ GETRESPONSETIME(m)
if $r_m \le r_{th}$ **then**
  $f_{v_s} \leftarrow$ GENERATEFLOWRULES(m, v_s, v_d) ▷ Fallback: route through main controller if it satisfies the delay bound
  return $\{f_{v_s}\}$
else
  return NULL ▷ No controller can serve the request within acceptable delay
end if

The sequence diagram in Fig. 2 contains four entities, which include vehicles, local controllers, domain controllers, and the main controller. When a local controller receives a route request, it checks if the source and the destination vehicle lie within its range. If this holds true, then the load status is determined using the above-lying associated domain controller. If the controller load status is found to be underloaded, the corresponding controller processes the route request. In the case of a controller being overloaded, a controller with minimum load is selected

Algorithm 2 Pseudocode to determine the load status of a controller.

Require: Controller c, response time threshold r_{th}
Ensure: Load-status s_c, response-time r_c
enum LoadStatus = {UNDERLOADED, NEUTRAL, OVERLOADED}
$r_c \leftarrow$ GETRESPONSETIME(c)
if $r_c > r_{th}$ **then**
    $s_c \leftarrow$ OVERLOADED
else if $r_c < r_{th}$ **then**
    $s_c \leftarrow$ UNDERLOADED
else
    $s_c \leftarrow$ NEUTRAL
end if
return (s_c, r_c)

Algorithm 3 Underloaded controller selection.

Require: Set of controllers $\mathbb{C}$, source vehicle v_s, response time threshold r_{th}
Ensure: Most underloaded controller c_{ul}, or NULL if none found
$\mathcal{U} \leftarrow \emptyset$ ▷ Set of associated controllers that are underloaded
$R \leftarrow \emptyset$ ▷ Map to store response times of candidate controllers
for all $c \in \mathbb{C}$ **do**
    if $\chi(v_s, c) =$ true **then** ▷ Source vehicle v_s is associated with controller c
        $(s_c, r_c) \leftarrow$ CLASSIFYCONTROLLER(c, r_{th})
        if $s_c =$ UNDERLOADED **then**
            $\mathcal{U} \leftarrow \mathcal{U} \cup \{c\}$
            $R[c] \leftarrow r_c$
        end if
    end if
end for
if $\mathcal{U} = \emptyset$ **then**
    return NULL
end if
$c_{ul} \leftarrow \arg\min_{c \in \mathcal{U}} R[c]$ ▷ Select associated underloaded controller with minimum response time
return c_{ul}

from the set of controllers and a path is determined between the two to forward the request to the underloaded controller. This underloaded controller processes the route request and generates the corresponding route request flow rule. Otherwise, if the source and destination do not lie within its vicinity, the domain controller processes the request and generates corresponding flow rules. Otherwise, if the local and domain controllers fail to generate the flow rule for the route request, the domain controller forwards the request to the main controller for processing and generating the flow rule for the same.

Having discussed the mechanism of our proposed routing scheme in Section 4, we provide a detailed discussion of the simulated results for our proposed routing scheme in Section 5.

5. Performance evaluation

This section presents the simulation environment and evaluates the performance of the proposed routing scheme in comparison with l_2-HSDVN and l_3-HSDVN, using both analytical modeling and simulation-based analysis to provide insights into flow setup delay, controller load computation, and the optimal number of control plane layers. Additionally, the proposed scheme is evaluated using key routing metrics such as average throughput, packet delivery ratio (PDR), end-to-end delay, and jitter, under varying vehicle densities and speeds, to assess the robustness and effectiveness of the proposed scheme in HSDVN.

Table 6
Simulation settings.

Parameter	*Value*
Simulation area	$3000m \times 1000m$
Number of vehicle nodes	20 - 200
Number of local,domain, main controllers	10, 2, 1
Maximum number of packets	1000
Hello packet time interval	5 s
Beacon packet time interval	5 s
Data packet generation	Poisson distribution
Data packet size	1024 bits
Mode of communication	V2V,V2I, I2I
V2V/V2I communication	DSRC
I2I communication	LTE
Number of simulation runs	10
DSRC data rate	6 Mbps
LTE data rate	500 Mbps
GPS accuracy	40 ns
Response time threshold	0.005 s
NS-version	NS-3.31
Communication loss model	Two-Ray Ground
MAC	802.11p
Vehicle speed	0–20 m/s
Simulation time	100 s

5.1. Simulation environment

To evaluate the performance of the proposed scheme, simulations were conducted using the ns-3 network simulator, integrated with the Simulation of Urban MObility (SUMO) tool for vehicular traffic generation. Realistic road topologies were extracted from OpenStreetMap (OSM) and imported into SUMO to generate mobility traces representing diverse urban traffic scenarios. These traces were then fed into ns-3 to simulate dynamic vehicular movement, where each vehicle acts as a data plane nodes in the SDVN. The control plane architecture comprising of local, domain, and main controllers was modeled using ns-3, enabling the simulation of flow setup delays, route request processing, and controller load-handling strategies. Parameters such as vehicle density and speed were varied to evaluate the performance of the proposed scheme under different network conditions and assess its routing efficiency in SDVN. Additional simulation settings are summarized in Table 6. To ensure statistical reliability, the simulation scenario was executed ten times, and the reported values represent the average across all runs. The variations across repetitions were minimal, confirming the consistency and stability of the simulation results.

5.2. Results and discussion

This section presents and discusses the simulation results obtained using ns-3, SUMO, and OSM-generated road topology. For each performance metric, comparative graph plots illustrate the behavior of the proposed scheme, l_2-HSDVN, and l_3-HSDVN across varying vehicle density and speed to validate the proposed scheme's ability in improving the routing performance correlated with theoretical findings to reinforce the analytical soundness of the proposed scheme. All performance metrics reported in the figures, represent the mean of ten independent simulation runs. Variability across runs was small, and this multi-run averaging ensures stability and reproducibility of the reported trends. The performance trends observed in the simulation results align with the analytical delay and load formulations presented in Section 3, particularly the predicted reduction in queuing delay and escalation penalties when controller load is balanced.

Fig. 3 illustrates the total effective delay with respect to the number of control planes. The total effective delay in this context refers to the cumulative latency experienced in the network due to flow setup time, inter-controller communication, and control packet propagation between the data plane's vehicles and control planes, controller pro-

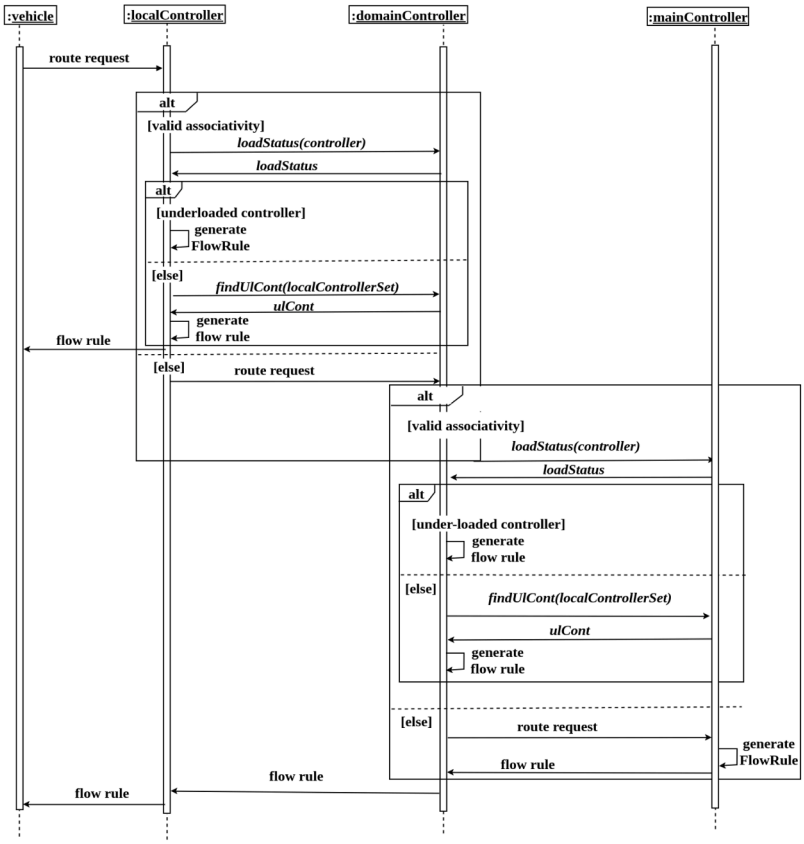


Fig. 2. Sequence diagram illustrating the route request processing and flow-rule generation in the l_3-HSDVN control plane.

cessing time, queuing delay, controller-to-controller message forwarding, and escalation penalties incurred when decisions must be pushed up to higher layers in a multi-tier HSDVN. The graph demonstrates that the delay is minimized when the number of control planes is three, indicated by the red annotation, *Optimal* $n = 3$. For $n < 3$, the control layer is likely too flat, leading to overloading of controllers and slower response times due to centralized bottlenecks. Conversely, beyond $n = 3$, the network becomes too hierarchical, and the excessive inter-layer communication and escalation overheads lead to steep increases in delay. This forms a convex curve, highlighting the non-linear trade-off. Thus, based on this analysis, we select $n = 3$ as the control plane configuration for all subsequent simulations, to ensure optimal routing performance with respect to latency.

Fig. 4 shows how the escalation penalty varies with respect to the number of control planes in H-SDVN. The escalation penalty quantifies the cumulative overhead introduced when local or intermediate-level controllers escalate decision-making tasks to higher-level controllers. This metric encompasses the additional time, communication cost, and synchronization delays incurred during such vertical inter-controller interactions. As evident from the plot, the escalation penalty grows slowly when the number of control planes is small but increases sharply beyond a certain threshold. The penalty is minimized at $n = 3$, marked in red as the optimal configuration. For $n < 3$, the control structure is overly centralized, resulting in fewer escalations but greater local processing load. In contrast, when $n > 3$, the hierarchical depth increases, leading to frequent and costlier decision escalations across multiple layers. This results in a steep, non-linear rise in the penalty value, forming a convex curve. Hence, minimizing the escalation penalty is vital for reducing responsiveness degradation in control-intensive tasks. Based on this observation, we fix $n = 3$ as the optimal control plane depth for subsequent simulations to balance decision autonomy and escalation overhead efficiently.

Fig. 5 presents a comparative analysis of queuing delay, escalation penalty, and total effective delay as functions of the number of control planes in HSDVN. The queuing delay, represented by the red curve, remains relatively constant and low across all values of n, indicating that queue-induced latency at individual controllers does not significantly vary with hierarchy depth. In contrast, the escalation penalty increases non-linearly as n increases. This reflects the growing cost of vertical decision propagation across multiple control layers, which becomes substantial beyond $n = 3$. The total effective delay aggregates all delay components including queuing delay, controller processing time, and escalation penalty, and follows a convex growth pattern. Initially, it decreases slightly or remains minimal due to load distribution across control layers, but it then increases sharply beyond the optimal point. Notably, the overlap between the escalation penalty and total effective delay curves beyond $n = 5$ suggests that escalation overhead becomes the dominant contributor to overall latency with increasing number if control planes. Therefore, this composite analysis underscores that minimizing control plane depth is critical to maintaining low-latency. Consistent with earlier findings, the optimal configuration remains at $n = 3$, which effectively balances load distribution and escalation avoidance. This value is thus used in all subsequent performance evaluations to ensure minimal delay overhead.

Fig. 6 illustrates the variation of individual delay components namely transmission delay, propagation delay, queuing delay, and escalation penalty with respect to the number of control planes in HSDVN. Among these, the transmission delay, propagation delay, and queuing delay remain relatively constant and low across the entire range of n. This consistency indicates that these delays are largely unaffected by the number of control layers, as they are more dependent on link capacity, physical distance, and local processing efficiency, respectively. In contrast, the escalation penalty increases non-linearly with n, dominating the delay landscape beyond $n = 4$. This rapid rise results from the grow-

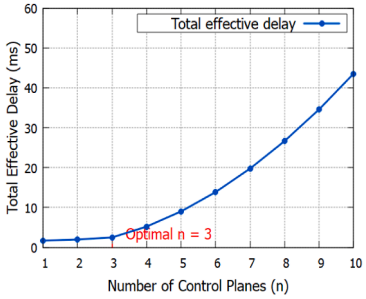


Fig. 3. Impact on total effective delay with increasing number of control planes.

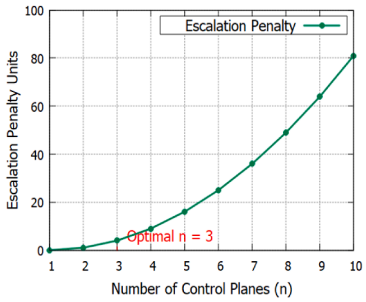


Fig. 4. Escalation penalty versus increasing number of control planes.

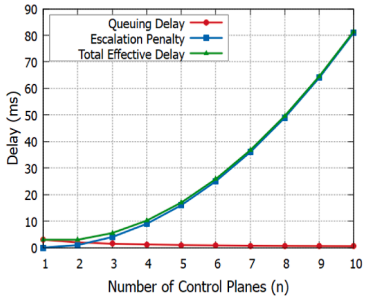


Fig. 5. Impact of increasing number of control planes on the delay behavior.

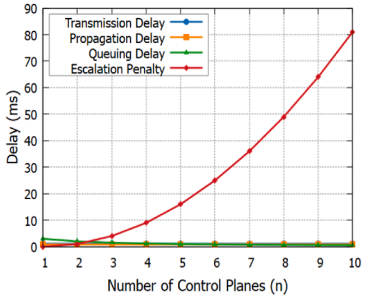


Fig. 6. Impact of increasing number of control planes on delay components.

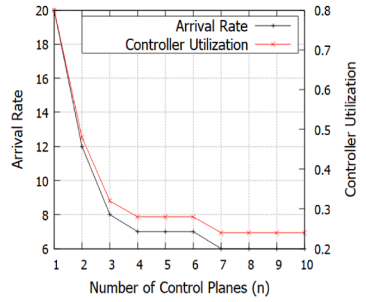


Fig. 7. Arrival rate and controller utilization per control plane.

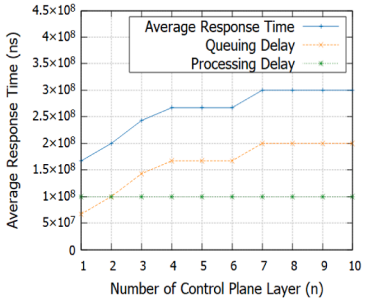


Fig. 8. Impact on average response time with varying number of control plane layer.

ing frequency and cost of vertical decision escalations as the network becomes deeper and more layered. The figure clearly shows that while fundamental communication delays remain stable, the hierarchical complexity primarily drives the increase in overall latency. Therefore, the graph plot reinforces the need to limit the number of control planes to mitigate excessive escalation overhead. Once again, this validates the choice of $n = 3$ as the optimal configuration for maintaining low control latency in HSDVN simulations.

Fig. 7 illustrates the relationship between the number of control planes and two key performance indicators such as the arrival rate of control requests and the controller utilization. The arrival rate refers to the average number of flow setup or control messages received per controller per unit time, while controller utilization quantifies the fraction of time the controller is actively processing these requests, modeled as $\rho = \lambda/\mu$, where λ is the arrival rate and μ is the service rate. As seen in the plot, both metrics sharply decrease as n increases from 1 to 3, reflecting a significant distribution of control load across multiple layers. This reduces the burden on individual controllers, mitigating queuing and processing delays. Beyond $n = 3$, both the arrival rate and controller utilization curves begin to flatten, indicating that adding more control layers does not significantly reduce the load on individual controllers. Instead, it introduces additional architectural complexity without offering meaningful improvements in load distribution. This behavior underscores the effectiveness of a three-layer control structure in balancing request load and maintaining controller efficiency. Hence, the observed flattening trend in both curves beyond $n = 3$ further supports the selection of $n = 3$ as the optimal configuration for ensuring better controller performance in the network.

Fig. 8 depicts how the average response time and its key components such as queuing delay and processing delay evolve with the number of control plane layers. The average response time represents the total time taken by a controller to respond to a flow setup request, encompassing both queuing and processing delays. As shown, the response time increases sharply from $n = 1$ to $n = 4$, and then gradually stabilizes beyond this point. This growth is primarily driven by the rise in queuing delay, which increases with additional control layers due to increased coordination and inter-layer dependencies. In contrast, the processing delay remains nearly constant across all values of n, indicating that the per-request computation cost at each controller is unaffected by the network's hierarchical depth. The stabilization in response time beyond $n = 6$ reflects a saturation point where added layers no longer significantly impact queuing behavior, but may still introduce unnecessary architectural complexity. These findings reinforce the earlier conclusion that a configuration with $n = 3$ or $n = 4$ strikes a balance between scalability and responsiveness, offering minimal delay without overburdening the control infrastructure.

Fig. 9 illustrates the variation in average response time experienced at the local and main control planes with respect to the number of vehicles in the network. As the number of vehicles increases from 20 to

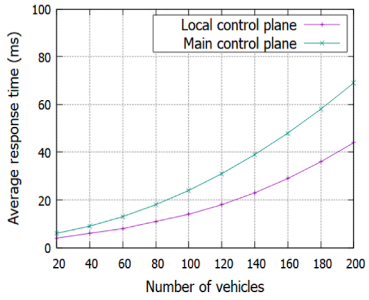


Fig. 9. Vehicle density versus average response time for l_2-HSDVN.

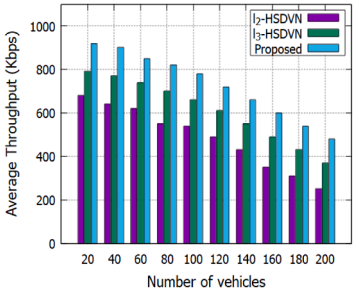


Fig. 11. Vehicle density versus average throughput.

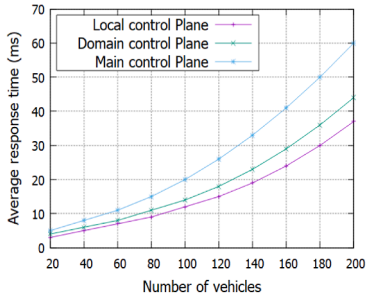


Fig. 10. Vehicle density versus average response time for l_3-HSDVN.

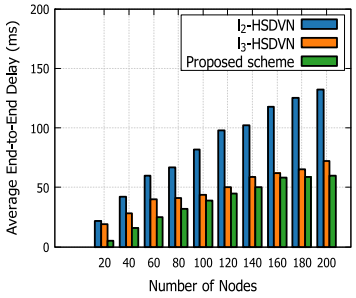


Fig. 12. Vehicle density versus average end-to-end delay.

200, both control planes exhibit a rising trend in response time due to the growing number of incoming route requests. However, the main control plane consistently exhibits significantly higher response times compared to the local control plane. This disparity is due to the centralized nature of the main controller, which is responsible for handling escalated or unresolved requests from lower layers, thereby experiencing higher queuing and processing loads. In contrast, the local control plane, being closer to the data plane, handles decisions with lower latency and limited overhead. The non-linear growth in response time at the main control plane suggests that as network density increases, the centralized approach becomes a bottleneck. This observation underscores the importance of hierarchical distribution, where more decision-making is delegated to the local control planes to maintain responsiveness under high traffic loads. It reinforces the design principle that offloading tasks to lower-tier controllers can significantly improve the system's scalability and reduce control latency in dense vehicular environments.

Fig. 10 presents the variation in average response time across three hierarchical levels of the control plane with respect to the number of vehicles in the network. As vehicle density increases from 20 to 200, the response time rises across all control layers due to the growing number of control requests and the resulting processing and queuing delays. Among the three, the local control plane consistently exhibits the lowest response time, benefitting from its proximity to the data plane and its ability to process frequent, low-complexity decisions rapidly. The domain control plane incurs higher response times as it handles more aggregated and potentially escalated requests. The main control plane, being at the top of the hierarchy and responsible for global decisions or conflict resolution, experiences the highest delay, especially under high traffic loads. The widening gap between the curves as vehicle count increases highlights the inefficiency of relying heavily on upper-tier controllers in dense environments. This trend emphasizes the effectiveness of hierarchical control plane, offloading most decision-making to local and domain controllers not only reduces the burden on the main controller but also ensures timely control decisions. These findings validate the hierarchical design choice and the critical role of the HSDVN in achieving scalable and delay-tolerant vehicular network control.

Fig. 11 illustrates the variation in average throughput with respect to vehicle density for the proposed routing scheme, in comparison with l_2-HSDVN and l_3-HSDVN. Among the three schemes, l_2-HSDVN consistently exhibits the lowest throughput, with a steady decline as vehicle density increases. This trend highlights its inefficiency in handling increasing number of route requests with increasing number of vehicles in the network. The l_3-HSDVN performs better than l_2-HSDVN by leveraging a hierarchical control approach. However, it still suffers from throughput degradation at higher vehicle densities, primarily due to the absence of a load-handling mechanism. The influx of route requests at the controllers may lead to controller overloading, increasing response times and a subsequent drop in overall throughput. In contrast, the proposed scheme achieves the highest throughput across all vehicle densities. While throughput does decrease with increase in vehicle density, the reduction is far less compared to the other two schemes because of the load-handling mechanism at the control plane, achieving approximately 53.96% higher throughput than l_2-HSDVN and 23.94% higher than l_3-HSDVN under the same network conditions. The marginal throughput decline is observed at high vehicle density. This is likely due to increased number of control packet exchanges required for flow rule generation, controller synchronization, and flow management across control layers, that leads to network congestion.

Fig. 12 depicts the graph for average end-to-end delay verses vehicle density for l_2-HSDVN, l_3-HSDVN, and the proposed scheme. The end-to-end delay refers to the total time taken for a data packet to travel from the source to the destination vehicle. As vehicle density increases, all three schemes exhibit an upward trend in end-to-end delay. Among them, l_2-HSDVN shows the highest delay, primarily due to a limited number of controllers responsible for handling all flow setup requests across the network. As the number of vehicles in the network increases, the rising number of route requests overloads the controllers, leading to longer controller response time. In contrast, l_3-HSDVN leverages a hierarchical control structure that distribute the controller load more evenly. The hierarchical structure allows local controllers to handle route requests, forwarding them to higher levels only when local processing is not possible, thus reducing processing and queuing delays. Although some delay is introduced through inter-controller communication, the HSDVN effectively prevents any single controller from becoming a bottleneck. This is evident in the plot, where the delay in l_3-HSDVN is consistently lower than in l_2-HSDVN

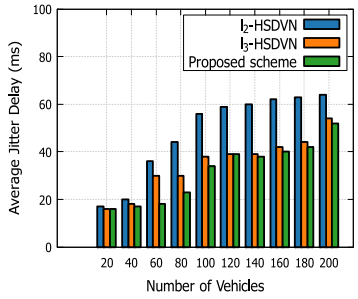


Fig. 13. Vehicle density versus average end-to-end jitter delay.

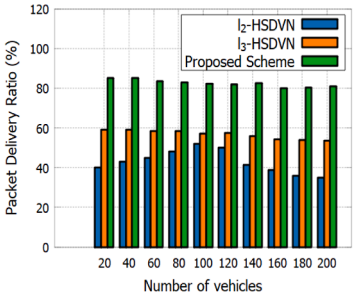


Fig. 14. Vehicle density versus packet delivery ratio.

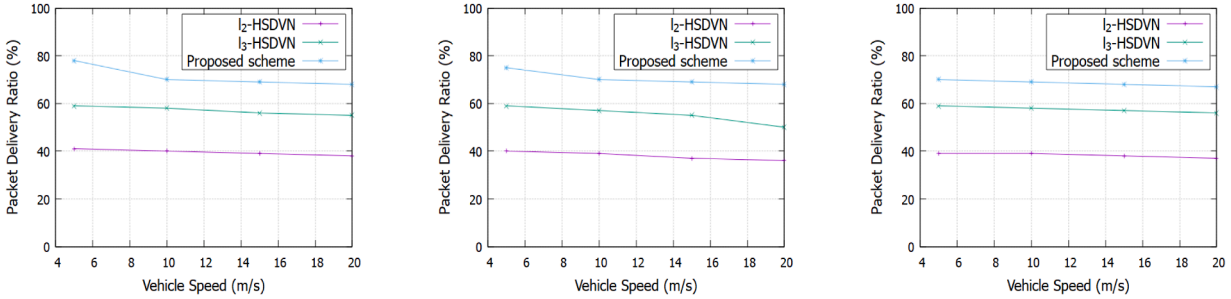
across different vehicle density. However, l_3-HSDVN lacks controller load-handling scheme. The proposed scheme demonstrates clear superiority over both l_2-HSDVN and l_3-HSDVN, by incorporating controller load-handling mechanism, actively monitoring controller utilization and redirecting requests from overloaded to underutilized controllers across the control plane. This adaptive load balancing prevents localized congestion and ensures timely flow setups, even under high vehicle densities. As a result, queuing and processing delays remain minimal, yielding significantly lower end-to-end delays across all traffic conditions. Notably, the proposed scheme achieves up to 56.48% improvement over l_2-HSDVN and 21.87% improvement over l_3-HSDVN without load-handling. The considerable reduction in delay achieved by LAERS compared to the l_2-HSDVN and l_3-HSDVN architectures can be attributed to two primary factors. Firstly, the dynamic load-aware redirection mechanism effectively reduces control-message congestion by preventing overloaded controllers from engaging in redundant flow-setup exchanges. This directly lowers the queuing and processing latency at each controller, leading to faster route-establishment times. Secondly, the hierarchical coordination employed in LAERS inherently improves route caching efficiency, as domain controllers maintain updated flow-state mappings that can be reused for subsequent route requests. This reuse minimizes route recalculation delays and further contributes to latency reduction. Together, these effects explain the more pronounced improvement observed over l_2-HSDVN, where controller congestion dominates the delay, and over l_3-HSDVN, where static inter-controller coordination limits caching benefits. These interpretations are consistent with findings in latency and energy-aware routing frameworks for multi-layer network architectures [55].

Fig. 13 illustrates the impact of vehicle density on the average jitter across three routing schemes: l_2-HSDVN, l_3-HSDVN, and the proposed scheme. Jitter refers to the variation in packet arrival delay over time. Excessive jitter can lead to out-of-order packet delivery and degraded routing performance. With increasing vehicle density, jitter increases across all three schemes. This can be attributed to several underlying factors. As vehicle density rises, network congestion increases due to a larger number of route requests, and multiple control packet exchanges, resulting in variable queuing delays at both the data and control planes. The l_2-HSDVN causes controller overload under high vehicle density scenario, leading to inconsistent processing times and higher jitter. Additionally, dynamic topology changes, disrupt established routes and trigger rerouting, causing delay variations between packets. Poor or fluctuating link quality further exacerbates the issue through retransmissions or buffering. Moreover, the absence of load balancing in l_2-HSDVN leads to uneven controller utilization, where some controllers become bottlenecks while others remain underutilized. Although l_3-HSDVN improves upon this by distributing control responsibilities hierarchically, it still suffers from jitter due to the lack of load handling scheme and the overhead of inter-controller communication across layers. The proposed scheme, however, adaptively redistributes requests away from overloaded controllers and minimizes controller-to-controller communication delays, significantly reducing delay variability. Consequently, the proposed scheme maintains the lowest jitter delay, showcasing its robustness and ability to provide stable communication in dynamic and high-density vehicular networks.

Fig. 14 illustrates the relationship between PDR and vehicle density for l_2-HSDVN, l_3-HSDVN, and the proposed scheme. As vehicle density increases, PDR declines across all three schemes due to rising network congestion. l_2-HSDVN exhibits the lowest PDR, particularly beyond 100 vehicles, where its centralized controller becomes overwhelmed, resulting in substantial packet drops. The absence of intermediate domain controllers limits its ability to scale under dense traffic conditions. l_3-HSDVN shows improved PDR over l_2-HSDVN by leveraging a hierarchical structure that distributes control traffic more evenly. However, without a dynamic load-handling mechanism, some controllers experience traffic surges, leading to local overload and increased packet loss, especially at densely populated vehicle network scenarios. In contrast, the proposed scheme consistently maintains the highest PDR compared to the other two schemes. Its load-aware redirection strategy ensures that route requests are handled by underloaded controllers, reducing control plane congestion and enabling timely flow rule generation. As a result, the proposed scheme sustains over 80% PDR. Compared to l_2-HSDVN and l_3-HSDVN without load-handling, the proposed scheme shows improvements of approximately 52.77% and 24.28%, respectively.

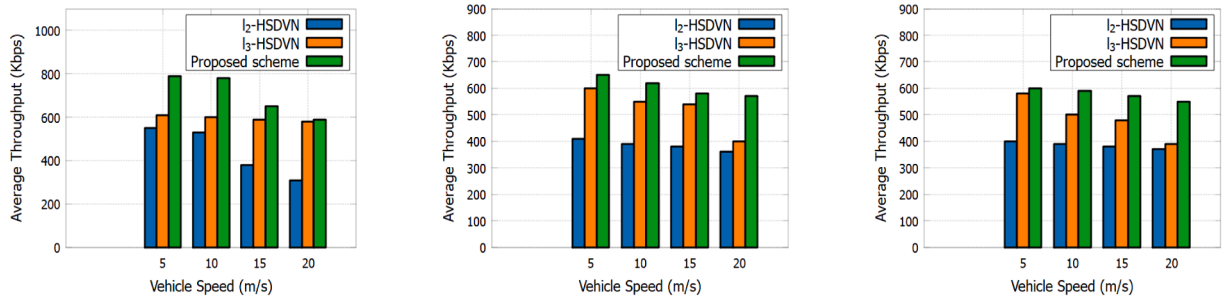
Fig. 15 illustrates the effect of increasing vehicle speed on PDR for l_2-HSDVN, l_3-HSDVN, and the proposed LAERS scheme across 50, 100, and 200 vehicles. In Fig. 15(a), with 50 vehicles, l_2-HSDVN records the lowest PDR, declining steadily as speed increases. This degradation arises from its reliance on a centralized controller, which struggles to maintain consistent flow rule delivery under dynamic topology changes. l_3-HSDVN performs better due to its hierarchical structure, but without load-awareness, it also suffers from reduced PDR at higher speeds. In contrast, LAERS maintains the highest PDR with only marginal degradation. Its dynamic load-handling mechanism ensures timely flow rule generation and stable packet delivery even under moderate mobility. Fig. 15(b), with 100 vehicles, shows more pronounced differences. l_2-HSDVN experiences a sharper drop in PDR, indicating worsening congestion and control delays. l_3-HSDVN also begins to show signs of controller overload as density and speed increase. The proposed scheme, however, continues to demonstrate high PDR due to its ability to redirect traffic to underloaded controllers, and minimizing packet loss. In Fig. 15(c), with 200 vehicles, all schemes show declining PDR with increasing speed. However, the decline is most severe in l_2-HSDVN, while l_3-HSDVN maintains moderate performance. The proposed scheme continues to outperform both schemes, with its decline in PDR remaining gradual. This robustness can be attributed to the integration of load-awareness within the l_3-HSDVN framework, which mitigates controller congestion and reduces packet drops despite high mobility and network load. Across all three scenarios, the proposed scheme demonstrates superior scalability and adaptability, maintaining significantly higher PDR compared to the baseline schemes.

Fig. 16, consisting of three sub-graphs, illustrates the impact of vehicle speed on average throughput for network scenarios with 50, 100, and 200 vehicles. Fig. 16(a) corresponds to the network scenario with



(a) For vehicles=50 (b) For vehicles=100 (c) For vehicles=200

Fig. 15. Impact of vehicle speed on packet delivery ratio.

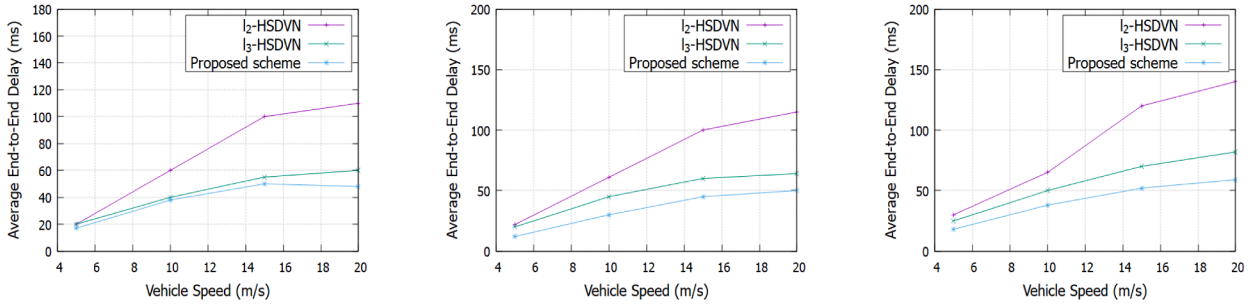


(a) For vehicles=50 (b) For vehicles=100 (c) For vehicles=200

Fig. 16. Impact of vehicle speed on average throughput.

50 vehicles. Here, l_2-HSDVN consistently shows the lowest throughput across all speeds, with a steady decline as speed increases. The limited two-layer control plane struggles to manage mobility-induced dynamics such as frequent packet losses, leading to reduced throughput. l_3-HSDVN, while performing better due to its hierarchical architecture, still lacks controller load-handling scheme. As vehicle speed increases, certain controllers become overloaded, resulting in performance degradation. In contrast, the proposed scheme achieves the highest throughput across all speeds. The integration of a dynamic load-handling mechanism enables balanced traffic distribution and minimizes congestion, thereby maintaining high throughput even at higher mobility levels. Fig. 16(b) represents the scenario with 100 vehicles. The limitations of l_2-HSDVN become more evident here, as the centralized controller architecture becomes a bottleneck under both high vehicle density and increasing speed, causing a notable drop in throughput. l_3-HSDVN continues to offer better throughput than l_2-HSDVN but suffers from a gradual decline due to the absence of adaptive traffic management. The proposed scheme outperforms both schemes, demonstrating strong resilience to increasing speed. Fig. 16(c) shows the behavior in a high-density network with 200 vehicles. As expected, l_2-HSDVN experiences a significant decline in throughput across all vehicle speeds. Although l_3-HSDVN performs better, it too shows substantial degradation due to control plane congestion and lack of adaptive load distribution. The proposed scheme maintains superior performance with the highest throughput across all speed levels because of dynamic load-handling mechanism that effectively mitigates the effects of mobility and congestion, leading to minimal throughput degradation even under heavy network load. Overall, across all three scenarios, the proposed scheme consistently demonstrates superior scalability and mobility resilience. While all schemes show some decrease in throughput with increasing vehicle speed, the decline in the proposed scheme is significantly less pronounced. This highlights the critical role of the load-handling mechanism in maintaining high throughput under varying traffic loads and mobility conditions. In contrast, the absence of such a mechanism in l_2-HSDVN and l_3-HSDVN results in substantial throughput loss as both vehicle density and speed increase, underscoring the importance of intelligent traffic distribution for efficient SDVN performance.

Fig. 17 includes three sub-graphs that depict the relationship between average end-to-end delay and vehicle speed for network scenarios with 50, 100, and 200 vehicles. Fig. 17 (a), with vehicle nodes as 50, depicts that l_2-HSDVN has the highest average delay across all speeds, and the delay increases significantly as vehicle speed rises. The l_3-HSDVN has a lower average delay compared to the l_2-HSDVN but increases with higher vehicle speeds. The l_3-HSDVN improves traffic management over l_2-HSDVN, but the absence of load-handling scheme results in controller overloading, causing operational delays as speed increases. The proposed scheme consistently exhibits better average end-to-end delay among the three schemes. Although there is a slight increase in delay with rise in vehicle speed, it performs better compared to the other two schemes. Fig. 17 (b), for 100 vehicles, l_2-HSDVN shows a further increase in delay as vehicle speed rises, leading to significant delays due to the inability to handle increased vehicle density network scenarios effectively. The l_3-HSDVN performs better than the l_2-HSDVN, but it still faces increased delays with increasing vehicle speed. The lack of load-handling scheme makes it challenging to manage traffic efficiently at higher vehicle density, causing delays as some controllers become overloaded. The propsed scheme performs best even with 100 vehicles and the increase in delay is minimal compared to the other schemes. Fig. 17 (c), for 200 vehicles, l_2-HSDVN exhibits a substantial increase in average end-to-end delay as vehicle speed increases, l_3-HSDVN performs better than l_2-HSDVN and faces noticeable delays as vehicle speed rises. The absence of load-handling becomes more detrimental under high vehicle density network conditions, leading to increased load at the control plane and operational delays. The proposed scheme consistently performs better even with 200 vehicles. While there is a slight increase in delay for higher speeds, it is much less pronounced compared to the other schemes. The load-handling capability ensures that traffic is handled well among controllers, minimizing delays and enhancing overall network performance. Across all three graphs, the proposed scheme demonstrates superior scalability and performance by



(a) For vehicles=50 (b) For vehicles=100 (c) For vehicles=200

Fig. 17. Impact of vehicle speed on average end-to-end delay.

maintaining lower average end-to-end delays compared to routing using l_2-HSDVN and l_3-HSDVN. The load-handling mechanism enables the proposed scheme to handle increased dense vehicle network scenario with increasing vehicle speed effectively. The absence of load-handling schemes in l_2-HSDVN and l_3-HSDVN leads to congestion and increased latency. As vehicle speed increases, all schemes experience an increase in end-to-end delay. However, the proposed scheme shows minimal delay, highlighting its effectiveness in maintaining low end-to-end delay under varying mobility conditions.

Fig. 18 presents the controller overhead incurred by l_2-HSDVN, l_3-HSDVN, and LAERS scheme across varying vehicular densities. Controller overhead quantifies the control-plane communication cost associated with network coordination among controllers and RSUs. It encompasses the volume of control packets exchanged for state monitoring, load reporting, route redirection, and flow-rule installation. In the simulation, this overhead is measured as the total amount of control packets transmitted and received through the designated control ports of each controller in NS-3. The cumulative control bytes exchanged during a simulation run are normalized over the simulation time to yield the overhead in kilobytes per second (kB/s), expressed as

$$\mathrm{OH} = \frac{1}{T}\sum_{c\in C}\left(B_c^{\mathrm{tx}} + B_c^{\mathrm{rx}}\right)\Big/1024, \tag{35}$$

where B_c^{tx} and B_c^{rx} denote the control bytes transmitted and received by controller c, respectively, and T is the simulation duration. This formulation, consistent with that adopted by Yusuf et al. [53], captures the true communication load imposed by the control operations rather than merely the number of packets exchanged. Fig. 18 illustrates the variation of controller overhead with vehicular density for the baseline l_2-HSDVN, l_3-HSDVN, and LAERS. As vehicular density increases, all schemes experience higher control overhead owing to increased monitoring and route-setup activities. However, the proposed LAERS mechanism consistently demonstrates the lowest overhead across all densities. The observed overhead for LAERS ranges from approximately 5.4 kB/s at 20 vehicles to 14.8 kB/s at 200 vehicles, whereas the corresponding values for l_2-HSDVN and l_3-HSDVN vary between 9.5-26 kB/s and 7.9–21.7 kB/s, respectively. This represents an average reduction of about 43% relative to l_2-HSDVN and 31% relative to l_3-HSDVN. The improvement may be because of the event-driven nature of LAERS, where control messages are triggered only when a controller exceeds its adaptive response-time threshold, avoiding the periodic and redundant status exchanges typical of conventional SDVN. This event-driven redirection behavior limits unnecessary control-plane message exchanges under rising traffic load, which is consistent with scalability requirements observed in federated frontline network frameworks [63].

To further validate the efficiency of the proposed scheme and to ensure a realistic and consistent comparison, the behavioral principles of the dynamic controller-placement schemes proposed by Toufga et al. [64] and Abdi Seyedkolaei et al. [65] were modeled within the same SUMO-NS3 co-simulation framework used for evaluating LAERS. In Toufga et al. [64], the ILP-based dynamic placement mechanism was emulated by periodically recomputing controller positions and RSU-controller associations based on vehicular density and inter-RSU hop distance, thereby capturing the traffic-aware reconfiguration behavior of the original model. Similarly, for [65], the Iterated Local Search principle was represented by adaptively reassigning controller-switch mappings based on real-time load variations, capturing the iterative reallocation behavior of the original model. Both [64,65] schemes operated under identical network parameters, mobility traces, and controller hierarchies as LAERS, ensuring fairness and methodological consistency throughout the evaluation. Beyond the baseline HSDVN architectures discussed in Figs. 14 and 15, the LAERS scheme was evaluated against two dynamic load-mitigation schemes, Toufga et al. [64] and Abdi Seyedkolaei et al. [65]. Under identical SUMO and NS-3 simulation settings, LAERS achieves an average Packet Delivery Ratio of 82.7%, compared with 68.1% for [64] and 68.7% for [65], corresponding to improvements of 21.4% and 20.3%, respectively. Throughput results exhibit a similar trend as in Figs. (11–16), where LAERS provides average gains of 21.2% over the [64] and 20.6% over [65]. Furthermore, LAERS achieves substantially lower end-to-end delay, reducing latency by approximately 32% relative to [64] and 29.5% relative to the [65] as given in Figs. (12–17). These improvements arise from LAERS's event-driven, response-time-based redirection mechanism, which continuously mitigates controller traffic. For completeness, the controller overhead was also compared with [64] and [65], where [64] and [65] exhibit overhead ranges of 7.5–20.6 kB/s and 7.3–20.1 kB/s, respectively, under identical conditions. LAERS maintains a significantly lower range of 5.4–14.8 kB/s, translating to average reductions of 28% and 32% relative to [64] and [65]. These results confirm that the latency benefits achieved by mitigating controller queu-

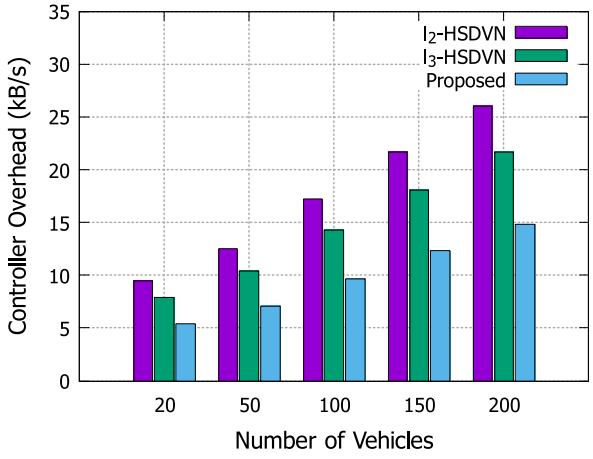


Fig. 18. Number of vehicles versus controller overhead.

ing delays consistently outweigh the redirection control packets cost, demonstrating that LAERS offers a superior control-plane efficiency, even under increasing vehicular density.

6. Conclusion

This paper presents LAERS, load-aware efficient routing scheme for l_3-HSDVN, that effectively addresses the key limitations of hierarchical SDVN architectures, such as high control plane operational delays, route discovery latency, controller overload, and poor scalability under dense vehicular traffic. By integrating a load-handling mechanism across local and domain controllers, LAERS intelligently redirects route requests away from overloaded controllers, thereby ensuring efficient load distribution and timely flow rule generation. Extensive NS-3 simulations validate the superiority of LAERS in diverse traffic conditions. The scheme achieves 54.46% and 19.93% improvements in average throughput, 52.77% and 24.28% gains in packet delivery ratio, and reductions of 56.48% and 21.87% in end-to-end delay compared to l_2-HSDVN and l_3-HSDVN, respectively. These results underscore the critical importance of incorporating load-awareness into the SDVN control plane to mitigate network congestion and maintain high-quality routing performance in dynamic vehicular environments. While LAERS offers significant advancements in traffic management and communication efficiency, this study assumes a static control plane and does not address controller failure scenarios. These assumptions follow standard practice in SDVN performance evaluation and were intentionally adopted to isolate and assess the core contribution of this work, the load-aware redirection mechanism, without additional external factors influencing the results. As part of future work, we aim to enhance the resilience and adaptability of LAERS by exploring dynamic controller placement strategies and incorporating controller failure recovery mechanisms, thereby further strengthening its applicability in real-world vehicular deployments. Beyond the evaluated scenarios, the hierarchical and distributed design of LAERS demonstrates strong scalability potential in heterogeneous vehicular environments. Its adaptive coordination enables effective operation under varying RSU densities and mixed communication technologies such as DSRC, C-V2X, and 5G. By dynamically balancing controller workloads in response to real-time network conditions, LAERS can maintain stable performance and efficient routing across diverse deployment scales, underscoring its practical suitability for future intelligent transportation systems.

CRediT authorship contribution statement

Madhuri Malakar: Writing - review & editing, Writing - original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Conceptualization; **Judhistir Mahapatro:** Writing - review & editing, Writing - original draft, Visualization, Validation, Supervision, Resources, Methodology, Investigation, Conceptualization; **K.L.K. Sudheera:** Writing - review & editing, Writing - original draft, Visualization, Validation, Supervision, Software, Methodology, Investigation; **Bibhudatta Sahoo:** Writing - review & editing, Writing - original draft, Validation, Supervision, Methodology, Investigation.

Data availability

No data was used for the research described in the article.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Proof of optimality

To analytically determine the optimal number of control layers, we consider the structure of the expected effective flow setup delay $\delta_{\text{effective}}(n)$. As discussed previously, the delay behavior exhibits a U-shaped curve due to the opposing effects of queuing delay and escalation penalty as the number of layers increases. Such behavior can be reasonably approximated by a convex quadratic function of the form:

$$\delta_{\text{effective}}(n) = an^2 + bn + c \tag{A.1}$$

where, n is the number of control plane layers, $a > 0$ represents the escalating growth of delay due to repeated penalties at higher layers, b models linear effects such as increasing propagation and minor processing overheads, and c captures the baseline network setup delay. The condition $a > 0$ ensures that the function is convex, implying the existence of a unique minimum. Differentiating $\delta_{\text{effective}}(n)$ with respect to n and equating to zero yields the stationary point and can be given as:

$$\frac{d}{dn}\delta_{\text{effective}}(n) = 2an + b = 0 \quad \Rightarrow \quad n^* = -\frac{b}{2a} \tag{A.2}$$

Furthermore, the second derivative confirms that the stationary point corresponds to a minimum and is given as:

$$\frac{d^2}{dn^2}\delta_{\text{effective}}(n) = 2a > 0 \tag{A.3}$$

Since typical network parameters yield small b relative to a, $n^* \approx 3$ minimizes $\delta_{\text{effective}}(n)$. Thus, mathematically and based on practical deployments, a three-layer control plane with local, domain, and main control planes efficiently balances load distribution, escalation penalty, and routing delays, achieving optimal performance.

References

[1] M.S.M. AL-inizi, O.M. NSAIF, Securing vehicle-to-vehicle communications: VANet challenges, Babylonian J. Netw. 2023 (2023) 105–111.

[2] B.A.A. Nunes, M. Mendonca, X.-N. Nguyen, K. Obraczka, T. Turletti, A survey of software-defined networking: past, present, and future of programmable networks, IEEE Commun. Surv. Tutorials 16 (3) (2014) 1617–1634. https://doi.org/10.1109/SURV.2014.012214.00180

[3] O. Sadio, I. Ngom, C. Lishou, Design and prototyping of a software defined vehicular networking, IEEE Trans. Veh. Technol. 69 (1) (2020) 842–850. https://doi.org/10.1109/TVT.2019.2950426

[4] M. Zhu, Z. Cai, J. Cao, M. Xu, Efficient multiple-copy routing in software-defined vehicular networks, in 2015 Int. Conf. Inf. Commun. Technol. (ICT IET (2015). https://doi.org/10.1049/cp.2015.0222

[5] R. Masoudi, A. Ghaffari, Software defined networks: a survey, J. Netw. Comput. Appl. 67 (2016) 1–25.

[6] M.M. Islam, M.T.R. Khan, M.M. Saad, D. Kim, Software-defined vehicular network (SDVN): a survey on architecture and routing, J. Syst. Architect. 114 (2021) 101961.

[7] X. Ji, H. Yu, G. Fan, W. Fu, SDGR: an SDN-based geographic routing protocol for VANET, in: 2016 IEEE Int. Conf. on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData), IEEE, 2016, pp. 276–281.

[8] L. Zhao, Z. Li, A.Y. Al-Dubai, G. Min, J. Li, A. Hawbani, A.Y. Zomaya, A novel prediction-based temporal graph routing algorithm for software-defined vehicular networks, IEEE Trans. Intell. Transp. Syst. 23 (8) (2022) 13275–13290. https://doi.org/10.1109/TITS.2021.3123276

[9] M. Zhu, J. Cao, D. Pang, Z. He, M. Xu, SDN-based routing for efficient message propagation in VANET, in: Wireless Algorithms, Systems, and Applications: 10th Int. Conf. , WASA 2015, Qufu, China, August 10–12, 2015, Proceedings 10, Springer, 2015, pp. 788–797.

[10] Y.-C. Liu, C. Chen, S. Chakraborty, A software defined network architecture for geobroadcast in VANETs, in: 2015 IEEE Int. Conf. on Communications (ICC), London, UK, IEEE, 2015, pp. 6559–6564. https://doi.org/10.1109/ICC.2015.7249370

[11] N.H. Hussein, S.P. Koh, C.T. Yaw, S.K. Tiong, F. Benedict, T. Yusaf, K. Kadirgama, T.C. Hong, SDN-based VANET routing: a comprehensive survey on architectures, protocols, analysis, and future challenges, IEEE Access 2024, https://doi.org/10.1109/ACCESS.2024.3355313).

[12] K.L.K. Sudheera, M. Ma, G.G.M.N. Ali, P. Han Joo Chong, Delay efficient software defined networking based architecture for vehicular networks, in: 2016 IEEE Int. Conf. on Communication Systems (ICCS), Shenzhen, China, IEEE, 2016, pp. 1–6. https://doi.org/10.1109/ICCS.2016.7833564

[13] H.M. Kavana, V.B. Kavya, B. Madhura, N. Kamat, Load balancing using SDN methodology, Int. J. Eng. Res. Technol. 7 (5) (2018) 206–208.

[14] Y. Zhang, L. Cui, W. Wang, Y. Zhang, A survey on software defined networking with multiple controllers, J. Netw. Comput. Appl. 103 (2018) 101–118.

[15] M. Karakus, A. Durresi, Quality of service (QoS) in software defined networking (SDN): a survey, J. Netw. Comput. Appl. 80 (2017) 200–218.

[16] S. Zhang, J. Lan, P. Sun, Y. Jiang, Online load balancing for distributed control plane in software-defined data center network, IEEE Access 6 (2018) 18184–18191. https://doi.org/10.1109/ACCESS.2018.2820148
[17] O.S. Oubbati, M. Atiquzzaman, P. Lorenz, A. Baz, H. Alhakami, SEARCH: an SDN-enabled approach for vehicle path-planning, IEEE Trans. Veh. Technol. 69 (12) (2020) 14523–14536. https://doi.org/10.1109/TVT.2020.3043306
[18] L. Zhao, J. Li, A. Al-Dubai, A.Y. Zomaya, G. Min, A. Hawbani, Routing schemes in software-defined vehicular networks: design, open issues and challenges, IEEE Intell. Transp. Syst. Mag. 13 (4) (2021) 217–226. https://doi.org/10.1109/MITS.2019.2953557
[19] B. Ali, Next-generation AI for advanced threat detection and security enhancement in DNS over HTTPS, Available at SSRN 5224919 (2025).
[20] Y. Liu, Y. Ge, Q. Meng, Q. Liu, Controller load optimization strategies in software-defined networking: a survey, J. Netw. Comput. Appl. 233 (2025) 104043.
[21] H. Ghafoor, I. Koo, CR-SDVN: a cognitive routing protocol for software-defined vehicular networks, IEEE Sens. J. 18 (4) (2017) 1761–1772.
[22] M. Chahal, S. Harit, Network selection and data dissemination in heterogeneous software-defined vehicular network, Comput. Netw. 161 (2019) 32–44.
[23] B. Dong, W. Wu, Z. Yang, J. Li, Software defined networking based on-demand routing protocol in vehicle ad HOC networks, in: 2016 12th Int. Conf. on Mobile Ad-Hoc and Sensor Networks (MSN), Hefei, China, IEEE, 2016, pp. 207–213. https://doi.org/10.1109/MSN.2016.041
[24] A. Kazmi, M.A. Khan, M.U. Akram, DeVANET: decentralized software-defined VANET architecture, in: 2016 IEEE Int. Conf. on Cloud Engineering Workshop (IC2EW), Berlin, Germany, IEEE, 2016, pp. 42–47. https://doi.org/10.1109/IC2EW.2016.12
[25] K.L.K. Sudheera, M. Ma, P.H.J. Chong, Link stability based optimized routing framework for software defined vehicular networks, IEEE Trans. Veh. Technol. 68 (3) (2019) 2934–2945.
[26] S.K.L. Kushan, M. Ma, P.H.J. Chong, Efficient flow instantiation via source routing in software defined vehicular networks, in: 2017 IEEE 86th Vehicular Technology Conf. (VTC-Fall), Toronto, ON, Canada, IEEE, 2017, pp. 1–5. https://doi.org/10.1109/VTCFall.2017.8288201
[27] W. Robert, M. Bounabi, A. Badr, Leveraging AI in mixed hierarchical topologies to improve WSN: a survey, Babylonian J. Netw. 2025 (2025) 59–69.
[28] F.X.A. Wibowo, M.A. Gregory, K. Ahmed, K.M. Gomez, Multi-domain software defined networking: research status and challenges, J. Netw. Comput. Appl. 87 (2017) 32–45.
[29] Y. Fu, J. Bi, K. Gao, Z. Chen, J. Wu, B. Hao, Orion: a hybrid hierarchical control plane of software-defined networking for large-scale networks, in: 2014 IEEE 22nd International Conference on Network Protocols, 2014, pp. 569–576. https://doi.org/10.1109/ICNP.2014.91
[30] Y. Fu, J. Bi, Z. Chen, K. Gao, B. Zhang, G. Chen, J. Wu, A hybrid hierarchical control plane for flow-based large-scale software-defined networks, IEEE Trans. Netw. Serv. Manage. 12 (2) (2015) 117–131. https://doi.org/10.1109/TNSM.2015.2434612
[31] L. Zhao, J. Hua, X. Ge, S. Zhong, Traffic engineering in hierarchical SDN control plane, in: 2015 IEEE 23rd International Symposium on Quality of Service (IWQoS), 2015, pp. 189–194. https://doi.org/10.1109/IWQoS.2015.7404732
[32] B. Genge, P. Haller, A hierarchical control plane for software-defined networks-based industrial control systems, in: 2016 IFIP Networking Conference (IFIP Networking) and Workshops, 2016, pp. 73–81. https://doi.org/10.1109/IFIPNetworking.2016.7497208
[33] S. Gopalakrishnan, E.D.K. Ruby, D. Hemanand, R. Anitha, D. Suresh, et al., Leveraging AI and blockchain in MANETs to enhance smart city infrastructure and autonomous vehicular networks, Babylonian J. Mach. Learn. 2024 (2024) 112–120.
[34] H. Trivedi, S. Tanwar, P. Thakkar, Software defined network-based vehicular ad-hoc networks for intelligent transportation system: recent advances and future challenges, in: Futuristic Trends in Network and Communication Technologies: First Int. Conf. , FTNCT 2018, Solan, India, 2018, Springer, 2019, pp. 325–337.
[35] W. Zhang, A. Festag, R. Baldessari, L. Le, Congestion control for safety messages in VANETs: concepts and framework, in: 8th Int. Conf. on ITS Telecommunications, Phuket, Thailand, 2008, pp. 199–203. https://doi.org/10.1109/ITST.2008.4740256
[36] L. Zhao, Z. Yin, K. Yu, X. Tang, L. Xu, Z. Guo, P. Nehra, A fuzzy logic-based intelligent multiattribute routing scheme for two-layered SDVNs, IEEE Trans. Netw. Serv. Manage. 19 (4) (2022) 4189–4200. https://doi.org/10.1109/TNSM.2022.3202741
[37] N. Lin, D. Zhao, L. Zhao, A. Hawbani, M. Guizani, N. Kumar, ALPS: an adaptive link-state perception scheme for software-defined vehicular networks, IEEE Trans. Veh. Technol. 72 (2) (2023) 2564–2575. https://doi.org/10.1109/TVT.2022.3214660
[38] L. Zhao, T. Zheng, M. Lin, A. Hawbani, J. Shang, C. Fan, SPIDER: a social computing inspired predictive routing scheme for softwarized vehicular networks, IEEE Trans. Intell. Transp. Syst. 23 (7) (2022) 9466–9477. https://doi.org/10.1109/TITS.2021.3122438
[39] A. Abugabah, A.A. Alzubi, O. Alfarraj, M. Al-Maitah, W.S. Alnumay, Intelligent traffic engineering in software-defined vehicular networking based on multi-path routing, IEEE Access 8 (2020) 62334–62342. https://doi.org/10.1109/ACCESS.2020.2983204
[40] G. Li, X. Wang, Z. Zhang, SDN-based load balancing scheme for multi-controller deployment, IEEE Access 7 (2019) 39612–39622.
[41] M.R. Belgaum, S. Musa, M.M. Alam, M.M. Su'ud, A systematic review of load balancing techniques in software-defined networking, IEEE Access 8 (2020) 98612–98636.
[42] A.A. Neghabi, N.J. Navimipour, M. Hosseinzadeh, A. Rezaee, Load balancing mechanisms in the software defined networks: a systematic and comprehensive review of the literature, IEEE Access 6 (2018) 14159–14178.
[43] O. Adekoya, A. Aneiba, M. Patwary, An improved switch migration decision algorithm for SDN load balancing, IEEE OJ-COMS 1 (2020) 1602–1613. https://doi.org/10.1109/OJCOMS.2020.3028971
[44] A. Pietrabissa, L.R. Celsi, F. Cimorelli, V. Suraci, F.D. Priscoli, A. Di Giorgio, A. Giuseppi, S. Monaco, Lyapunov-based design of a distributed wardrop load-balancing algorithm with application to software-defined networking, IEEE Trans. Control Syst. Technol. 27 (5) (2018) 1924–1936.
[45] W.-K. Lai, Y.-C. Wang, Y.-C. Chen, Z.-T. Tsai, TSSM: time-sharing switch migration to balance loads of distributed SDN controllers, IEEE Trans. Netw. Service Manag. 19 (2) (2022) 1585–1597.
[46] Z. Latif, K. Sharif, F. Li, M.M. Karim, S. Biswas, M. Shahzad, S.P. Mohanty, DOLPHIN: dynamically optimized and load balanced path for inter-domain SDN communication, IEEE Trans. Netw. Service Manag. 18 (1) (2020) 331–346.
[47] M. Malakar, J. Mahapatro, T. Ghosh, A survey on routing and load-balancing mechanisms in software-defined vehicular networks, Wireless Netw. 30 (2024) 3181–3197.
[48] Y. Gao, Z. Zhang, D. Zhao, Y. Zhang, T. Luo, A hierarchical routing scheme with load balancing in software defined vehicular ad hoc networks, IEEE Access 6 (2018) 73774–73785.
[49] L. Alouache, T. Sylla, L. Mendiboure, H. Aniss, A fuzzy approach for load balancing in blockchain-based software defined vehicular networks, in: 2022 18th Int. Conf. on Wireless and Mobile Computing, Networking and Communications (WiMob), Thessaloniki, Greece, IEEE, 2022, pp. 235–242. https://doi.org/10.1109/WiMob55322.2022.9941696
[50] L. Mendiboure, M.A. Chalouf, F. Krief, Load-aware and mobility-aware flow rules management in software defined vehicular access networks, IEEE Access 8 (2020) 167411–167424.
[51] Z. He, D. Zhang, S. Zhu, J. Cao, X. Liu, SDN Enabled high performance multicast in vehicular networks, in: 2016 IEEE 84th Vehicular Technology Conf. (VTC-Fall), Montreal, QC, Canada, 2016, pp. 1–5. https://doi.org/10.1109/VTCFall.2016.7881215
[52] S. Correia, A. Boukerche, R.I. Meneguette, An architecture for hierarchical software-defined vehicular networks, IEEE Commun. Mag. 55 (7) (2017) 80–86.
[53] N.M. Yusuf, K.A. Bakar, B. Isyaku, A. Abdelmaboud, W. Nagmeldin, Controller placement with critical switch aware in software-defined network (CPCSA), PeerJ Comput. Sci. 9 (2023) e1698.
[54] W.H. Ghasab, A.A. Hadi, A.G.K. Alshami, A.D. Radhi, R.M. Al-Amri, A.H. Mousa, Construction a virtual central network for all Iraq's fire station, Babylonian J. Int. Things 2024 (2024) 151–160.
[55] F.A. Al-Ibraheemi, M.T. Tally, S. Nadweh, I.A.M. Al Sayed, J.F. Tawfeq, H.M. Gheni, P. Shah, R. Sekhar, A cognitive energy-driven routing strategy for ultra-efficient data transfer in wireless sensor networks, Appl. Data Sci. Anal. 2025 (2025) 131–143.
[56] S. Bera, S. Misra, M.S. Obaidat, Mobi-flow: mobility-aware adaptive flow-rule placement in software-defined access network, IEEE Trans. Mobile Comput. 18 (8) (2019) 1831–1842. https://doi.org/10.1109/TMC.2018.2868932
[57] J. Bhatia, Y. Modi, S. Tanwar, M. Bhavsar, Software defined vehicular networks: a comprehensive review, Int. J. Commun. Syst. 32 (12) (2019) e4005.
[58] N. Cardona, E. Coronado, S. Latré, R. Riggio, J.M. Marquez-Barja, Software-defined vehicular networking: opportunities and challenges, IEEE Access 8 (2020) 219971–219995.
[59] S. Rehman, N. Kapoor, A review on delay efficient architecture for software defined vehicular networks (SDVN), in: 2019 6th Int. Conf. on Computing for Sustainable Global Development (INDIACom), New Delhi, India, 2019, pp. 1094–1100.
[60] J. Cui, Q. Lu, H. Zhong, M. Tian, L. Liu, A load-balancing mechanism for distributed SDN control plane using response time, IEEE Trans. Netw. Serv. Manag. 15 (4) (2018) 1197–1206.
[61] B. Ali, G. Chen, Proactive and privacy-preserving defense for DNS over HTTPS via federated AI attestation (PAFA-DoH), Neural Netw. (2025) 108343.
[62] B. Ali, G. Chen, Neuromorphic quantum adversarial learning (NQAL): a bio-inspired paradigm for DNS over HTTPS threat detection, EURASIP J. Inf. Secur. 2025 (1) (2025) 35.
[63] B. Ali, On the fog's frontline: a federated machine learning approach for industrial network threat detection and intrusion prevention, J. Cybersecur. 11 (1) (2025) tyaf041.
[64] S. Toufga, S. Abdellatif, H.T. Assouane, P. Owezarski, T. Villemur, Towards dynamic controller placement in software defined vehicular networks, Sensors 20 (6) (2020) 1701.
[65] A. Abdi Seyedkolaei, S.A. Hosseini, Seno, A. Moradi, Dynamic controller placement in software-defined networks for reducing costs and improving survivability, Trans. Emerging Telecommun. Technol. 32 (1) (2021) e4152.